



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**

ДИПЛОМНА РАБОТА

СИСТЕМА ЗА ОТЧИТАНЕ НА ПРИСЪСТВИЯ НА СТУДЕНТИ В ТУ-СОФИЯ

Дипломант:

Мартин Дамянов

ф-н № 121222041

Специалност: Компютърно и софтуерно инженерство

Научен ръководител:

доц. д-р инж. Георги Цочев

София

2026

Съдържание

I.	УВОД	4
II.	Цели и задачи. Анализ на съществуващи разработки	5
1.	Цели на проекта	5
2.	Задачи на проекта	5
3.	Анализ на съществуващи разработки.....	6
3.1.	Системи базирани на RFID	6
3.2.	Платформени решения	6
3.3.	NFC/HCE разработки.....	7
4.	Обосновка на избрания подход	7
III.	Проектиране на системата.....	8
1.	Системна архитектура.....	8
2.	Компоненти.....	8
2.1.	ESP32 + PN532 (NFC четец)	8
2.2.	Android приложение	10
2.3.	FastAPI Backend	10
2.4.	Уеб интерфейс.....	10
3.	Описание на комуникационния поток.....	10
4.	Видове потребители на системата	12
5.	Модел на базата данни	13
5.1.	Users	14
5.2.	Students.....	14
5.3.	Teachers	14
5.4.	Courses.....	14
5.5.	Groups.....	14
5.6.	GroupCourses	14
5.7.	Schedules	15
5.8.	Attendance	15
6.	Проектиране на сигурността	15
IV.	Използвани технологии	16
1.	Хардуерни компоненти.....	16
1.1.	ESP32.....	16
1.2.	PN532 (NFC модул)	16

2.	Програмни езици	16
2.1.	C++ (ESP32 фърмуер).....	16
2.2.	Kotlin	17
2.3.	Python	17
2.4.	JavaScript.....	17
3.	Фреймуъркове и библиотеки.....	17
3.1.	FastAPI	17
3.2.	SQLAlchemy	17
3.3.	Adafruit PN532.....	18
3.4.	Retrofit2 (Android).....	18
4.	База данни.....	18
5.	Сигурност и криптография	18
5.1.	HMAC-SHA256	18
5.2.	Android Keystore и EncryptedSharedPreferences	18
5.3.	JSON Web Tokens.....	19
6.	Стандарти и протоколи	19
6.1.	ISO 14443 и ISO-DEP.....	19
6.2.	ISO 7816-4.....	19
6.3.	Host Card Emulation	19
V.	Програмна реализация	20
1.	Фърмуер на ESP32	20
1.1.	Конфигурация	20
1.2.	Визуална и звукова сигнализация	20
1.3.	Инициализация.....	21
1.4.	Основен цикъл.....	22
1.5.	Декодиране на получения payload.....	22
1.6.	Комуникация с бекенда	23
2.	Android мобилно приложение	24
2.1.	Съхранение на данните	24
2.2.	Комуникация с бекенд сървър	26
2.3.	Клас MainActivity.....	27
2.4.	Клас HomeActivity	28
2.5.	Клас NfcHceService.....	29

3.	Сървърна част	31
3.1.	models.py	31
3.2.	auth.py	31
3.3.	attendance.py	34
3.4.	administration.py	41
3.5.	main.py	41
4.	Уеб интерфейс.....	42
4.1.	Глобално състояние	42
4.2.	Функция api().....	43
4.3.	Вход в системата	43
4.4.	Навигация и страничен панел.....	44
4.5.	Зареждане на данни	45
4.6.	Разделяне на страници.....	46
4.7.	Модален прозорец.....	46
4.8.	Toast известия.....	47
VI.	Ръководство на потребителя	48
1.	Системни изисквания	48
2.	Инсталиране и конфигуриране.....	48
2.1.	Инсталиране на сървърната част	48
2.2.	Конфигурация и флашване на ESP32	49
2.3.	Инсталиране на мобилно приложение	49
3.	Работа с уеб интерфейса	49
3.1.	Вход в системата.....	49
3.2.	Работа с администраторски профил.....	50
3.3.	Работа с преподавателски профил	52
3.4.	Работа със студентски профил	52
4.	Работа с мобилно приложение	53
4.1.	Първоначално стартиране и вход в приложението	53
4.2.	Чекиране с NFC.....	53
5.	Често срещани проблеми	55
VII.	Заклучение.....	56
	Речник на използвани съкращения и термини	58
	Източници	59

I. УВОД

Отчитането на студентско присъствие в Технически университет - София по принцип се извършва ръчно: преподавателят дава присъствен лист, а студентите се записват в него, след което следва проверка. Друг вариант е преподавателя да пита поименно студентите, за да им отбележи присъствието ръчно в списък или електронна таблица. Тази практика е тровава, има възможности за фалшификация и не позволява централизирано съхранение и анализ на данните. При група от 25–30 студенти процедурата може да отнеме до 5 минути от всяко занятие.

Near Field Communication (NFC) и Host Card Emulation (HCE) - технологии, вградени стандартно в съвременните Android смартфони - предлагат ефективно решение. Чрез HCE телефонът действа като виртуална смарт карта и отговаря на криптографски команди от NFC четец, без да е нужен допълнителен хардуер от страна на студента, нито пък постоянна връзка с интернет или Bluetooth.

Настоящият дипломен проект - „Система за отчитане на присъствия на студенти в ТУ-София“ - проектира и реализира прототип на такава система. Тя обхваща: хардуерен NFC четец (ESP32 + PN532), Android приложение с HCE услуга и HMAC-SHA256 криптографска защита, RESTful бекенд (FastAPI + SQLite) с автоматично отчитане на присъствия в занятия по седмичния график на студента и уеб интерфейс за администратори, преподаватели и студенти.

Системата е защитена срещу replay атаки чрез Application Transaction Counter (ATC) и съхранява чувствителните ключове в Android Keystore. Разработката е тествана в локална мрежова среда и покрива пълния функционален цикъл - от регистрация на студент до визуализация на присъствията.

Документът е организиран в пет глави: анализ на съществуващи решения и цели на проекта, архитектурно проектиране, използвани технологии, програмна реализация и ръководство за потребителя.

II. Цели и задачи. Анализ на съществуващи разработки

1. Цели на проекта

Основната цел на настоящия дипломен проект е да се проектира и реализира функциониращ прототип на автоматизирана система за отчитане на студентско присъствие, базирана на NFC технология и Host Card Emulation (HCE). Системата трябва да замени ръчния процес по отбелязване на присъствията с автоматизирано решение. То трябва да е надеждно, бързо и криптографски защитено и приложимо в реална учебна среда.

Постигането на тази цел изисква покриване на пълния технологичен стек: хардуерен NFC четец в залата, мобилно приложение на студента и сървърна инфраструктура за съхранение и управление на данните.

2. Задачи на проекта

За постигане на поставената цел са формулирани следните конкретни задачи:

- Проектиране на сигурен бинарен NFC протокол между ESP32 четеца и Android HCE услугата, включващ HMAC-SHA256 верификация и Application Transaction Counter (ATC) за защита срещу replay атаки.
- Проектиране на сигурен бинарен NFC протокол между ESP32 четеца и Android HCE услугата, включващ HMAC-SHA256 верификация и Application Transaction Counter (ATC) за защита срещу replay атаки.
- Изграждане на хардуерен NFC четец на базата на ESP32-WROOM32 микроконтролер и PN532 модул в SPI режим. Четецът трябва да активира ISO-DEP (ISO 14443-4) сесия, да обменя APDU команди с телефона и да препраща верифицирания payload към сървъра по Wi-Fi.
- Изграждане на хардуерен NFC четец на базата на ESP32-WROOM32 микроконтролер и PN532 модул в SPI режим. Четецът трябва да активира ISO-DEP (ISO 14443-4) сесия, да обменя APDU команди с телефона и да препраща верифицирания payload към сървъра по Wi-Fi.

- Разработка на уеб интерфейс за управление на студенти, преподаватели, разписания и присъствия, достъпен за три роли: администратор, преподавател. Всяка роля има съответните за нея права за достъп.
- Тестване на системата в локална мрежова среда и проверка на пълния функционален цикъл.

3. Анализ на съществуващи разработки

Преди да се пристъпи към проектирането на системата, е направен преглед на съществуващите решения за автоматизирано отчитане на присъствие. Анализираните подходи могат да се групират в три категории: RFID-базирани системи (предшественик на настоящата разработка), платформени решения за управление на обучението и академични NFC проекти.

3.1. Системи базирани на RFID

Системите за отчитане на присъствие, базирани на RFID карти, са най-широко разпространеното хардуерно решение в академична среда. При тях всеки студент притежава физическа карта с уникален идентификатор (UID). Тя се допира до чекиращо устройство при влизане в залата. Четецът (обикновено RC522 или подобен) разпознава UID и го изпраща към централен сървър. [1]

Предимствата на RFID са очевидни: простота, ниска цена и висока надеждност на хардуера. Съществен недостатък обаче е пълната липса на криптографска защита — UID на картата се предава в открит вид и може да бъде лесно клониран с леснодостъпен хардуер (напр. Proxmark3). Освен това физическите карти се губят, повреждат и изискват административна поддръжка. [2]

3.2. Платформени решения

Google Classroom и Microsoft Teams for Education предлагат базова функционалност за управление на учебния процес, включително отбелязване на присъствие чрез ръчно въвеждане или интеграция с трети страни. Тези платформи са силно ориентирани към дистанционно обучение и не разполагат с механизъм за физическа верификация на присъствие в зала. [3] [4]

Основният им проблем в контекста на настоящия проект е, че се използват само за онлайн среда. Допълнително, те изискват постоянна интернет свързаност и лицензионни разходи.

3.3. NFC/HCE разработки

В академичната литература съществуват редица публикации, описващи системи за отчитане на присъствие чрез NFC. Характерно за повечето от тях е използването на пасивни NFC тагове (стикери), залепени в залата - студентът докосва тага с телефона и приложението записва присъствие. Този подход е по-евтин от RFID четците, но носи съществен проблем: тагът може да бъде разчетен и използван повторно, а местоположението му - симулирано дистанционно. [5] [6]

По-напредналите разработки използват двупосочна NFC комуникация (подобно на настоящия проект), но рядко включват криптографска верификация на ниво HMAC и ATC replay защита. Проектът в настоящата дипломна работа надгражда тези подходи, като добавя пълен криптографски протокол, интеграция с разписание и многоролев уеб интерфейс. [7] [8]

4. Обосновка на избрания подход

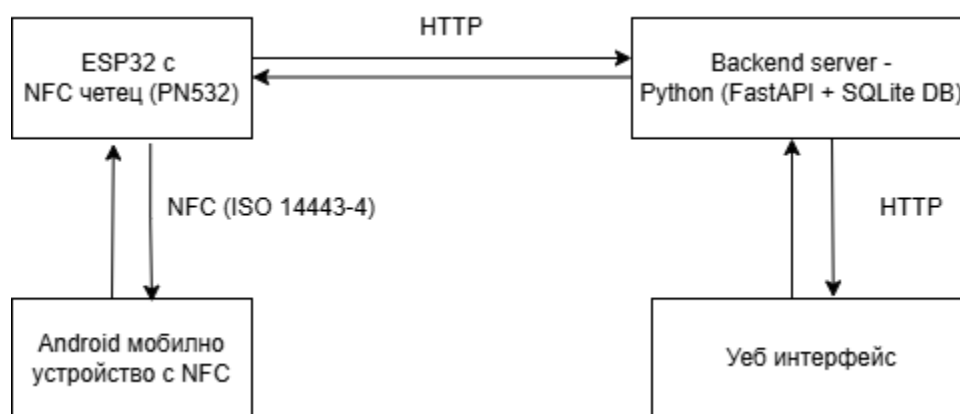
Въз основа на анализа е избрана архитектура, базирана на Android HCE и ESP32 с NFC четец, по следните съображения:

- Голяма част от студентите вече притежават NFC-съвместим смартфон – отпада необходимостта от издаването на RFID карти.
- HCE позволява пълен контрол върху криптографския протокол на ниво приложение, за разлика от пасивните RFID карти.
- ESP32 е евтин, широко достъпен и разполага с вградена Wi-Fi свързаност, което опростява инфраструктурата.
- HMAC-SHA256 с ATC осигурява криптографска сигурност, сравнима с банкови карти, без нужда от PKI(public key infrastructure) инфраструктура.
- Open-source стекът (FastAPI, SQLite, PlatformIO) спестява разходи за лицензи и позволява пълна адаптация към нуждите на ТУ-София.

III. Проектиране на системата

1. Системна архитектура

Системата е проектирана като трислойна клиент-сървър архитектура с четири основни компонента: хардуерен NFC четец, Android мобилно приложение, RESTful бекенд и уеб интерфейс. Компонентите комуникират по два независими канала: NFC (безжична близкодистанционна) между телефона и четеца, и HTTP/Wi-Fi между четеца и сървъра. На фиг. 1 е показана блокова схема на системното ниво.



Фиг. 1. – Блокова схема на системата

2. Компоненти

2.1. ESP32 + PN532 (NFC четец)

Хардуерният четец се реализира с микроконтролер ESP32-WROOM32, свързан с NFC модул PN532 чрез SPI интерфейс. Четецът непрекъснато слуша за NFC устройства чрез `inListPassiveTarget()` - метод, който активира ISO 14443-4 (ISO-DEP) протокол, необходим за APDU комуникация с Android HCE. При засичане на телефон изпраща SELECT AID APDU команда и в отговор получава криптиран payload. Полученият payload се преработва във формат с HEX, който FastAPI сървърът очаква. Преработения payload се препраща към сървъра като HTTP POST заявка.

На фиг. 2 е показана електрическата схема на свързване. Към ESP32 са свързани NFC сензорът PN532 чрез SPI интерфейс, един зелен и един червен светодиода, както и зумер. Към последните три елемента са свързани и резистори със съпротивление 150Ω с цел ограничаване на тока през веригите на елементите. Тези съпротивления са избрани след решаване на следното уравнение:

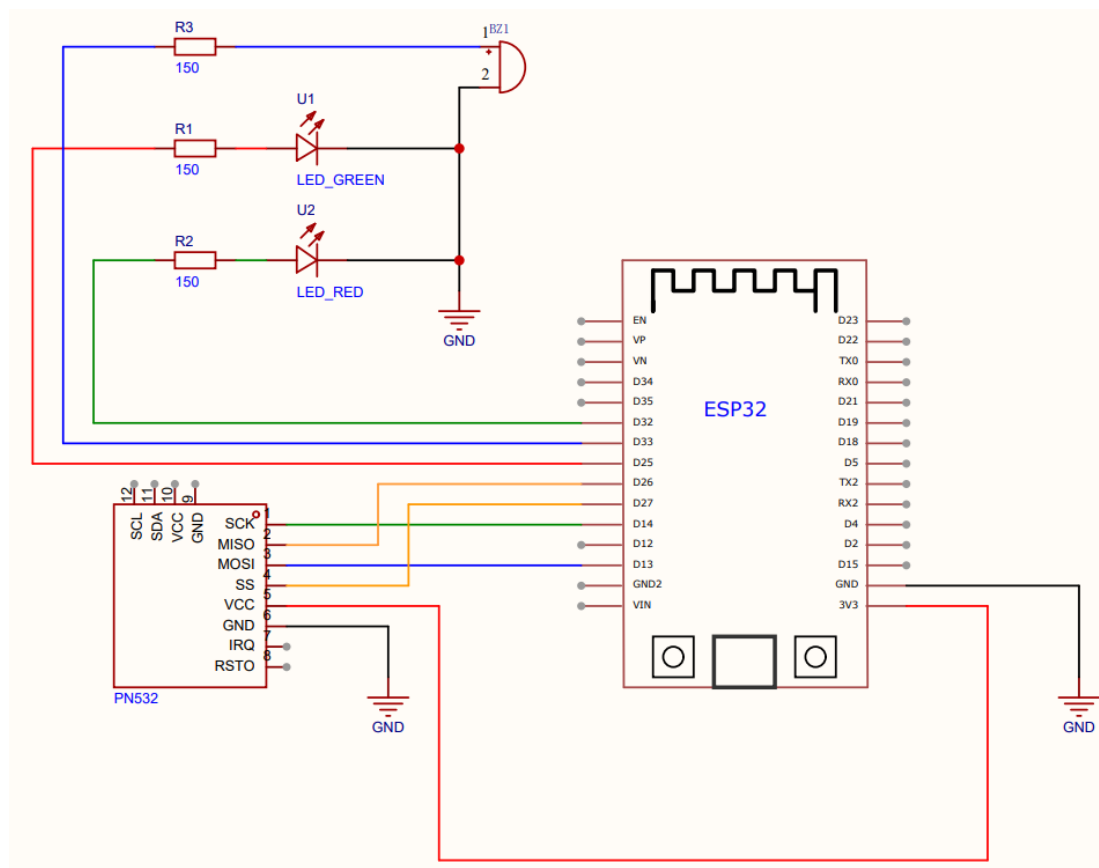
$$R = \frac{V_{cc} - V_{led}}{I_{led}}$$

Където:

- $V_{cc} = 3.3\text{ V}$
- V_{led} е работно напрежение на светодиода ($\approx 1.9\text{V}$ за червения и $\approx 2.0\text{V}$ за зеления светодиод)
- I_{led} е желаният ток през светодиодите

За продължителна и безопасна експлоатация на светодиодите, зумера и изводите на микроконтролера е препоръчително да не се надвишава стойността от 10mA ток, то за съпротивление на резисторите е избрана стойността 150Ω. По този начин тока във веригите може да бъде изчислен чрез горното уравнение. А стойностите на тока през съответните светодиоди са:

- През зелен светодиод: $\approx 8.6\text{mA}$
- През червен светодиод $\approx 9.3\text{mA}$



Фиг. 2. – Електрическата схема на свързването

2.2. Android приложение

Мобилното приложение се състои от четири компонента: *MainActivity*, *HomeActivity*, *NfcHceService* и *StudentPrefs*.

MainActivity обработва логването и получаването на JWT токен, HMAC ключ и начална стойност на АТС от сървъра, след което ги записва локално в криптиран вид. При следващо стартиране, ако данните вече са налични, екранът за вход се пропуска, тъй като данните вече са запаметени локално.

HomeActivity е главният екран - показва факултетния номер на студента и статуса на NFC. Чекирането се активира ръчно с бутон и е ограничено до 30-секунден времеви прозорец. След изтичане на времето, сесията се спира автоматично.

NfcHceService е HCE услугата, която отговаря на NFC четеца. При активна сесия конструира криптографски подписан бинарен payload и го изпраща при докосване.

StudentPrefs е криптирано хранилище (*EncryptedSharedPreferences* с *Android Keystore*), което пази HMAC ключа, факултетния номер и АТС брояча.

2.3. FastAPI Backend

Сървърът е реализиран с Python FastAPI и асинхронен SQLAlchemy, използващ SQLite. Организиран е в три рутера: *auth* (логин, предоставяне на HMAC, регистрации на потребители), *attendance* (чекиране, CRUD операции на присъствия) и *administration* (дисциплини, разписания, групи). Всички endpoints са защитени с JWT Bearer токени и оторизиране чрез роли.

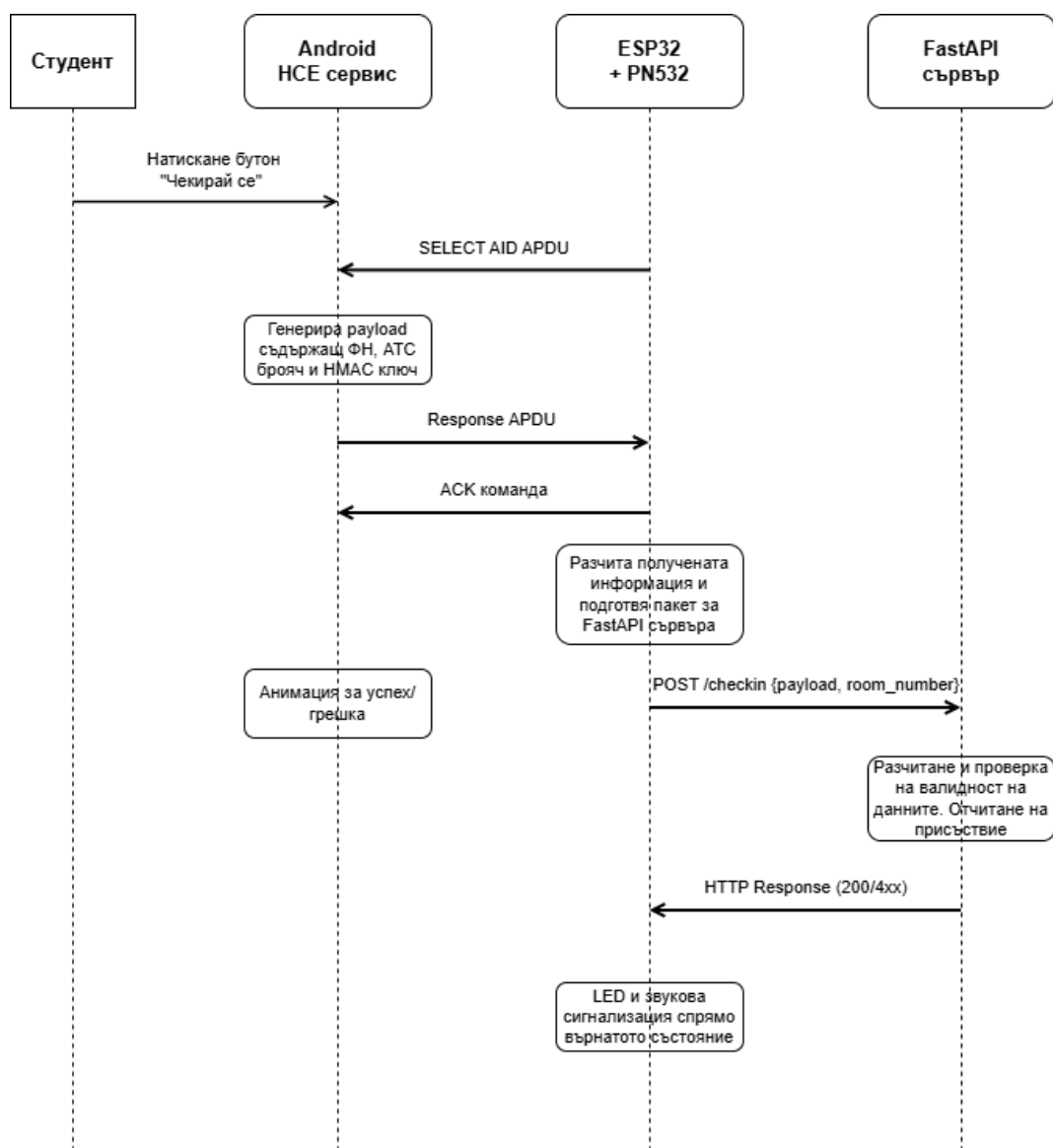
2.4. Уеб интерфейс

Фронтендът е реализиран като Single Page Application с Vanilla JS, без фреймуърк. Визуализира различни секции спрямо ролята на влезлия потребител. Комуникира с бекенда чрез относителни URL адреси, като FastAPI доставя статичните файлове директно. Интерфейсът е в цветовата гама на ТУ-София – тъмно синьо и бяло.

3. Описание на комуникационния поток

Комуникацията при NFC чекиране преминава през два физически канала - NFC между телефона и четеца, и HTTP/Wi-Fi между четеца и сървъра [10]. На фиг. 3 е показана диаграма на последователността от действия при чекиране на студент.

Бинарният формат на payload-а от мобилното приложение е проектиран с фиксирана дължина на HMAC частта (32 байта), тъй като Adafruit PN532 библиотеката разполага с ограничен буфер. Факултетния номер е с динамичен размер, поради което се предава и неговата дължина. Payload-а включва и АТС брояча. ESP32 декодира бинарния отговор и реконструира стринга "ФН|АТС|hmachex" в HEX формат, който пък е подходящ за FastAPI бекенда.



Фиг. 3. Sequence Diagram – последователност на действията при чекиране

След като студентът е натиснал бутона "Чекирай се" в мобилното приложение, комуникацията се реализира в следната последователност :

1. ESP32 засича телефона чрез `inListPassiveTarget()`, което активира ISO-DEP (ISO 14443-4) сесия. След това изпраща SELECT AID APDU команда.

2. NfcHceService на мобилното приложение разпознава AID-а, увеличава АТС брояча и изчислява HMAC-SHA256 чрез функция на факултетния номер на студента и АТС брояча . Създава бинарен буфер съдържащ HMAC ключа, факултетния номер и АТС брояча и го изпраща към чекиращото устройство.
3. ESP32 получава буфера и го парсва, като прочита факултетния номер, АТС брояча и конвертира 32-те HMAC байта в hex низ. Тази информация я преработва и я подготвя в нов текстовия низ в HEX формат, подходящ за FastAPI сървър.
4. ESP32 изпраща потвърдителна команда обратно към телефона. NfcHceService я разпознава, деактивира сесията за NFC и се показва анимация за успех. Ако NFC връзката прекъсне преди потвърждението за получаване, се показва анимация за грешка.
5. ESP32 изпраща POST /checkin към FastAPI с JSON тяло, съдържащо факултетния номер, АТС брояча и HMAC ключа.
6. FastAPI проверява дали АТС брояча е в очакваните граници спрямо този, записан в базата данни и дали HMAC-SHA256 съвпада. При успех, започва проверка за активно занятие по зала, ден и час, записва или отказва присъствието и връща HTTP отговор.
7. ESP32 получава отговора от сървър - при HTTP 200 активира зеления светодиод и звуков сигнал, при грешка – червения светодиод и звуков сигнал за грешка.

4. Видове потребители на системата

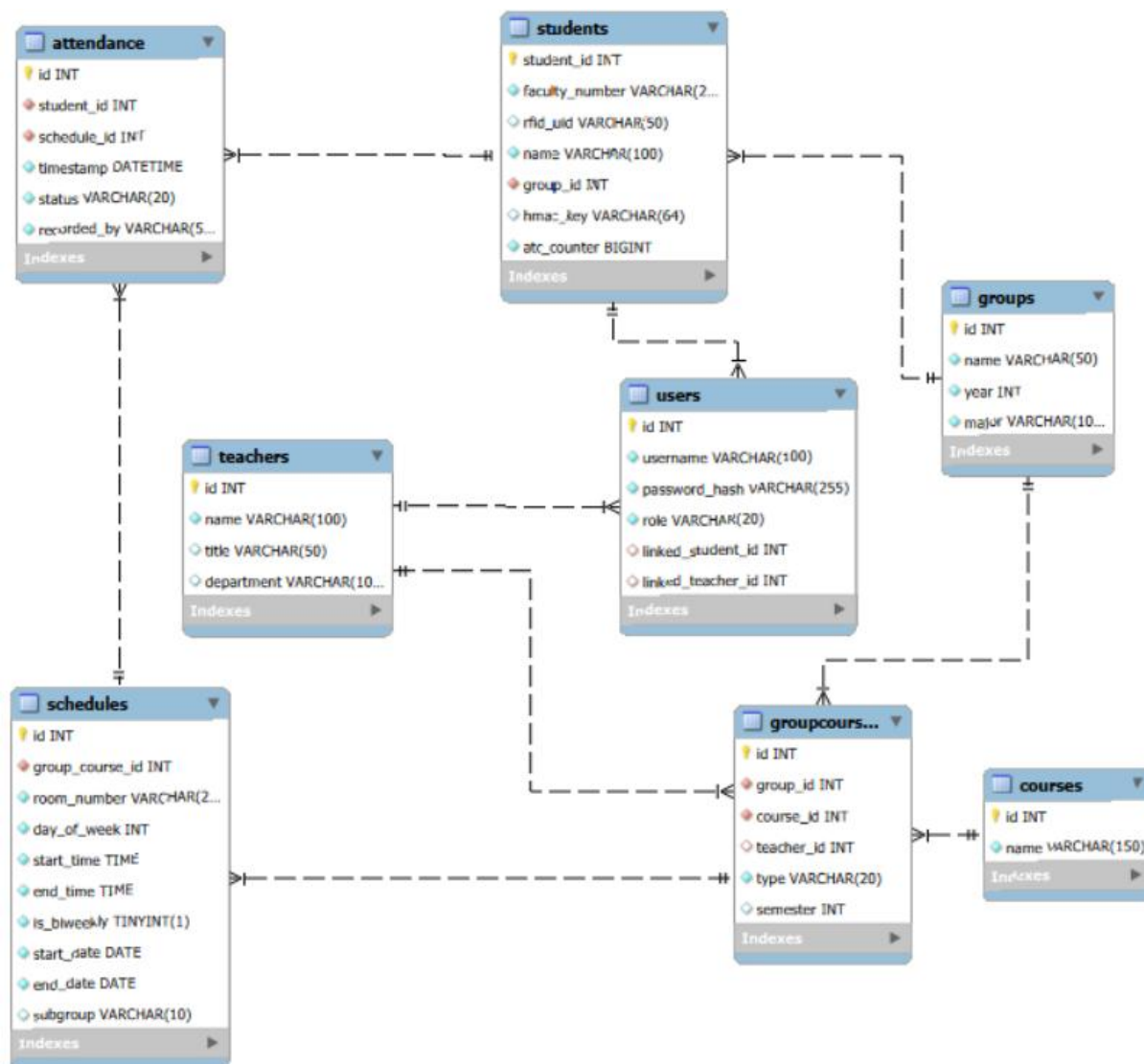
Системата обслужва три типа потребители с различни права:

- Администратор – има пълен достъп до уеб интерфейса: управление на студенти, преподаватели, групи, дисциплини, разписания, присъствия и регистриране на нови потребители.
- Преподавател – преглед на присъствията на студентите в преподаваните от него часове. Ръчно добавяне, редакция и изтриване на присъствия на студентите в неговите часове.

- Студент – Преглед само на собствените си присъствия. Използва един акаунт за достъп както в мобилното приложение, така и в уеб интерфейса.

5. Модел на базата данни

Базата данни е релационна SQLite и включва шест основни таблици: Students, Users, Groups, Courses, GroupCourses, Schedules и Attendance. На фиг. 4 е показана ER-диаграма на базата данни.



Фиг. 4. ER-диаграма на базата данни

В следващите точки са разгледани таблиците с тяхната структура и функция за проекта.

5.1. Users

Това е таблица, в която се съхраняват акаунтите на всички потребители на системата – студенти, преподаватели и администратори. Съдържа потребителско име, хеширана версия на паролата и роля (student, teacher, admin). Свързва се с Students или Teachers чрез foreign keys – по този начин един потребителски акаунт е обвързан с конкретен студент или преподавател.

5.2. Students

Таблица за студентите. Освен основните данни (факултетен номер, име, група), съдържа и полетата специфични за NFC системата: hmac_key (криптографският ключ за верификация) и atc_counter (монотонният брояч за защита срещу replay атаки). Полето rfid_uid е наследено от предишната RFID версия на системата и е незадължително.

5.3. Teachers

Съдържа данните на преподавателите: пълно име, академично звание и катедра. Използва се за свързване на преподавател към конкретно занятие в учебния план.

5.4. Courses

Таблица, съдържаща всички дисциплини в системата. Съдържа само наименованието на дисциплината — без група, преподавател или тип. Самостоятелна таблица, за да може една дисциплина да се преизползва за различни групи и типове занятия.

5.5. Groups

Съдържа учебните групи в университета. Всяка група се идентифицира с комбинация от наименование, курс и специалност. Служи като основна организационна единица, към която са свързани студентите и учебният план.

5.6. GroupCourses

Свързваща таблица между групи и дисциплини, която представлява учебния план. Определя кой преподавател води дадената дисциплина за дадена група и какъв е типът на занятието (лекция, упражнение, лаборатория). Уникалният ключ по (group_id, course_id, type) гарантира, че не може да се добави дублиран запис.

5.7. Schedules

Таблица за разписанията на занятията. За всеки запис в GroupCourses дефинира конкретния ден от седмицата, началния и крайния час, залата и периода на провеждане (начална и крайна дата). Поддържа двуседмично повторение (is_biweekly) и подгрупи (subgroup). Тази таблица е ключова за автоматичното отчитане на присъствията чрез чекирането с NFC.

5.8. Attendance

В тази таблица се записват всички присъствия. Всеки запис свързва студент с конкретно занятие от разписанието, съдържа точен timestamp на чекирането, статус (Присъствие, Отработване, Извинено) и кой е записал присъствието - Автоматичен при NFC чекиране или името на преподавателя, ако е вкарано ръчно.

6. Проектиране на сигурността

В системата са зададени няколко цели и изисквания за сигурността срещу съответните възможни заплахи:

- Аутентикация: JWT Bearer токени с 24-часов живот, издавани при успешен логин. Всеки endpoint на бекенд сървър е защитен с Depends(get_current_user). [10]
- NFC верификация: HMAC-SHA256 подпис върху факултетния номер и АТС брояча ("FN|ATC"). Ключът е 32 случайни байта, генерирани от os.urandom(), предава се на мобилното приложение само веднъж по HTTPS при първо логване. [16]
- Replay защита: Application Transaction Counter (ATC) е монотонен брояч. Сървърът приема само $ATC > DB_ATC$ и $ATC \leq DB_ATC + 100$. Прозорецът от 100 компенсира неуспешни допирания, при които телефонът увеличава АТС, но сървърът не записва присъствие. [11]
- Защита от дублиране: за всеки студент, предмет и тип занятие се допуска само едно присъствие на ден (проверка по course_id, type и date).
- Съхранение на ключове: на Android устройството HMAC ключът се съхранява в EncryptedSharedPreferences, поддържано от Android Keystore. Това е хардуерно изолирано хранилище, което е недостъпно дори при root достъп.

IV. Използвани технологии

Следва да бъдат описани технологиите, инструментите и библиотеките, използвани при разработката на системата. За всяка технология е посочена кратка характеристика и обосновка за нейния избор.

1. Хардуерни компоненти

1.1. ESP32

Използва се микроконтролер ESP32 Devkit V1, базиран на ESP32-WROOM32. Той е двудрен микроконтролер с тактова честота до 240 MHz, вградена Wi-Fi (802.11 b/g/n) и Bluetooth свързаност, произведен от Espressif Systems. Разполага с 38 GPIO пина, множество хардуерни интерфейси (SPI, I2C, UART) и 4 MB Flash памет.

Изборът на ESP32 пред алтернативи като Arduino Uno или Raspberry Pi е обоснован с три фактора: вградена Wi-Fi свързаност (без допълнителен модул), достатъчна изчислителна мощност за обработка на бинарни NFC payload-и и ниска цена (~5 USD).

1.2. PN532 (NFC модул)

PN532 е интегрална схема за NFC комуникация на NXP Semiconductors, [12] поддържаща стандартите ISO 14443-A/B и ISO 18092. Поддържа три интерфейса: UART, I2C и SPI. В проекта е конфигуриран в SPI режим чрез DIP превключватели (SEL0=OFF, SEL1=ON). Този режим е избран, тъй като ESP32 има готови библиотеки за комуникация по SPI и комуникацията му е по-бърза спрямо I2C.

PN532 е избран пред RC522 поради едно основно предимство: поддръжка на ISO-DEP (ISO 14443-4) протокол, което му позволява да комуникира с мобилни устройства с Android, които имат вграден NFC модул.[13]

2. Програмни езици

2.1. C++ (ESP32 фърмуер)

Фърмуерът на ESP32 е написан на C++ в рамките на Arduino Framework, управляван чрез PlatformIO. Arduino Framework предоставя абстрактен слой над хардуера, който позволява лесно програмиране на GPIO, SPI и Wi-Fi. Има готова библиотека Adafruit PN532, чрез която се използва и програмира NFC сензора.

2.2. Kotlin

Kotlin е официалният препоръчан език за Android разработка от Google от 2019 г. насам. Предлага пълна съвместимост с Java, по-кратък синтаксис, вграден null-safety и поддръжка на асинхронни операции. Kotlin е избран пред Java поради тъй като вече се е наложил като основен език за програмиране на Android приложения и има интеграция с Jetpack библиотеките.

2.3. Python

За реализацията на сървърната страна е използван Python 3.14. Избран е заради големият брой библиотеки за уеб разработка, ORM и криптография, бърза итерация и по-лесен синтаксис. За уеб фреймуърк е избран FastAPI, тъй като е по-нова и модерна технология, която позволява асинхронна обработка. Макар да не е паралелна, това подобрява производителността, особено ако постъпят заявки от много студенти наведнъж. За базите данни се използва SQLAlchemy.

2.4. JavaScript

Фронтендът е реализиран с Vanilla JavaScript (ES2022) без допълнителен фреймуърк. HTML5 и CSS3 осигуряват структурата и визията на фронтенда, а JS обработва комуникацията с REST API и динамичното обновяване на съдържанието.

3. Фреймуъркове и библиотеки

3.1. FastAPI

FastAPI е Python уеб фреймуърк за изграждане на REST API, базиран на стандарта OpenAPI и Pydantic за валидация на данните. Поддържа асинхронно изпълнение чрез Python asyncio и има възможност за автоматично генериране на документация. [18]

FastAPI е избран пред Flask и Django поради възможността за асинхронна обработка, автоматичната валидация на данните чрез Pydantic схеми и значително по-високата производителност при I/O операции.

3.2. SQLAlchemy

SQLAlchemy е ORM (Object-Relational Mapper) за Python, използван в асинхронен режим чрез разширението aiomysql. Дефинира моделите на базата данни като Python класове и генерира SQL заявки автоматично.

Асинхронният режим се използва, за да не блокира FastAPI event loop при изпълнение на бавни I/O операции към базата данни.

3.3. Adafruit PN532

Adafruit PN532 е C++ библиотека за управление на PN532 NFC модула в Arduino среда. Предоставя функции за инициализация, конфигурация и обмен на данни с NFC устройства. Ключово ограничение на версия 1.3.1 е вътрешният буфер от 64 байта. Това определя максималния размер на NFC payload-a.

3.4. Retrofit2 (Android)

Retrofit2 е типобезопасен HTTP клиент за Android и Java, разработен от Square. Позволява дефиниране на REST API endpoints като Kotlin интерфейси с анотации и поддържа асинхронно изпълнение. Retrofit е стандартният избор за мрежова комуникация в Android.

4. База данни

За съхранение на данните е използвана SQLite. Тя е олекотена релационна база данни, която не изисква отделен сървърен процес. Достъпът се осъществява асинхронно чрез драйвера aiosqlite, съвместим с asyncio event loop на FastAPI.

SQLite е избрана поради лесното внедряване – използва се един .db файл. Производителността му е достатъчна за проекта (стотици студенти) и пълната съвместимост с SQLAlchemy ORM. При нужда от мащабиране SQLAlchemy позволява миграция към PostgreSQL или MySQL с минимални промени в кода.

5. Сигурност и криптография

5.1. HMAC-SHA256

HMAC (Hash-based Message Authentication Code) с SHA-256 хеш функция се използва за криптографска верификация на NFC payload-a. Алгоритъмът изчислява функция върху съобщението "ФН|АТС", използвайки споделен таен ключ. [14] [15] [16]

5.2. Android Keystore и EncryptedSharedPreferences

Android Keystore System е хардуерно изолирано хранилище за криптографски ключове, достъпно чрез Android API. EncryptedSharedPreferences използва AES256-GCM за криптиране на стойностите и AES256-SIV за ключовете, управлявани от Keystore. [17]

По този начин HMAC ключът не може да бъде извлечен дори при root достъп до устройството, тъй като дешифрирането изисква хардуерна атестация от самия Keystore чип.

5.3. JSON Web Tokens

JSON Web Tokens (JWT) се използва за аутентикация в REST API. При успешен логин сървърът издава токен, подписан с HMAC-SHA256 и таен ключ, с 24-часов живот. Токенът се включва в Authorization header на всяка последваща заявка. JWT е предпочетен поради съвместимостта му с FastAPI, и лесната интеграция с други Python библиотеката.

6. Стандарти и протоколи

6.1. ISO 14443 и ISO-DEP

ISO 14443 дефинира физическия и протоколния слой за безконтактни карти с обхват до 10 cm. Частта ISO 14443-3 дефинира инициализацията и механизъм против колизии, докато ISO 14443-4 (ISO-DEP) добавя транспортен протокол за пренос на APDU команди. ISO-DEP е задължителен за комуникация с Android HCE услуги и се активира на ESP32 чрез функцията inListPassiveTarget() на Adafruit PN532 библиотеката. [19]

6.2. ISO 7816-4

ISO 7816-4 дефинира структурата на APDU (Application Protocol Data Unit) командите, използвани за комуникация между четеща и виртуалната карта. Командата APDU съдържа полета CLA, INS, P1, P2, Lc (дължина на данни) и данните. [20]

6.3. Host Card Emulation

Host Card Emulation (HCE) е функционалност, въведена в Android 4.4 (API 19), позволяваща на Android приложения да емулират NFC смарт карти без хардуерен Secure Element. Приложението регистрира AID идентификатор в apduservice.xml и Android OS маршрутизира съответните APDU команди към регистрираната HostApuService. [20]

V. Програмна реализация

Четвърта глава описва програмната реализация на системата. Системата се състои от четири компонента — фърмуер за ESP32, Android мобилно приложение, сървърна част и уеб интерфейс. За всеки компонент са представени основните класове и функции с показани части от кода.

1. Фърмуер на ESP32

Фърмуерът на ESP32 е реализиран на C++ в рамките на Arduino Framework. Той не извършва криптографска верификация - ролята му е да инициира NFC комуникацията, да декодира бинарният пакет, получен от телефона при чекирането, и да го препрати към FastAPI сървър посредством Wi-Fi. Цялата бизнес логика е в бекенд сървър.

1.1. Конфигурация

Параметрите за свързване към WIFI мрежата са дефинирани в отделен файл config.h като константи. GPIO пиновете са избрани така, че да не са в конфликт с вградените функции и запазени по подразбиране за други цели в ESP32. SELECT_AID_APDU също е записан като масив от константи. (фиг. 5)

```
#define PIN_PN532_SCK 14
#define PIN_PN532_MOSI 13
#define PIN_PN532_MISO 26
#define PIN_PN532_SS 27
#define PIN_LED_GREEN 25
#define PIN_LED_RED 32
#define PIN_BUZZER 33
#define BUZZER_CHANNEL 0

// Структура (ISO 7816-4):
// CLA INS P1 P2 Lc [AID - 7 байта] Le
// 0x00 0xA4 0x04 0x00 0x07 0xA0 0x00 0x00 0x02 0x47 0x10 0x01 0x00
static const uint8_t SELECT_AID_APDU[] = {
    0x00, 0xA4, 0x04, 0x00, 0x07,
    0xA0, 0x00, 0x00, 0x02, 0x47, 0x10, 0x01,
    0x00
};
```

Фиг. 5. Конфигурация на пиновете и SELECT_AID_APDU

1.2. Визуална и звукова сигнализация

Към микроконтролера са свързани два светодиода и пиезо зумер за визуална и звукова обратна връзка към студента. Зелен светодиод и двутонов сигнал показват, че чекирането е минало успешно, докато червен светодиод и нискотонов сигнал показват че има някаква грешка. Зумерът се управлява чрез ШИМ (LEDC периферия на ESP32) за генериране на различни честоти. Кодът е показан на фиг. 6:

```

void signalSuccess() {
    digitalWrite(PIN_LED_GREEN, HIGH);
    ledcWriteTone(BUZZER_CHANNEL, 1000);
    delay(150);
    ledcWriteTone(BUZZER_CHANNEL, 1400);
    delay(200);
    ledcWriteTone(BUZZER_CHANNEL, 0);
    delay(1800);
    digitalWrite(PIN_LED_GREEN, LOW);
}

void signalError() {
    digitalWrite(PIN_LED_RED, HIGH);
    ledcWriteTone(BUZZER_CHANNEL, 300);
    delay(700);
    ledcWriteTone(BUZZER_CHANNEL, 0);
    delay(1300);
    digitalWrite(PIN_LED_RED, LOW);
}

```

Фиг. 6. Код за управление на светодиодите и зумера

1.3. Инициализация

Функцията `setup()` се изпълнява еднократно при стартиране на микроконтролера. Инициализира GPIO пиновете, LEDC PWM канала за зумера, PN532 модула и Wi-Fi връзката. При неуспешна инициализация на PN532 или Wi-Fi устройството влиза в безкраен цикъл с мигащ светодиод, който сигнализира за проблем. (фиг. 7)

```

void setup() {
    Serial.begin(115200);
    pinMode(PIN_LED_GREEN, OUTPUT);
    pinMode(PIN_LED_RED, OUTPUT);
    ledcSetup(BUZZER_CHANNEL, 1000, 8);
    ledcAttachPin(PIN_BUZZER, BUZZER_CHANNEL);
    digitalWrite(PIN_LED_GREEN, LOW);
    digitalWrite(PIN_LED_RED, LOW);

    nfc.begin();
    if (!nfc.getFirmwareVersion()) {
        Serial.println("PN532 не е намерен!");
        while (true) { digitalWrite(PIN_LED_RED, !digitalRead(PIN_LED_RED)); delay(300); }
    }
    nfc.SAMConfig();
    nfc.setPassiveActivationRetries(0xFF);
    Serial.println("PN532 OK");

    Serial.printf("Свързване %s", WIFI_SSID);
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    for (int i = 0; i < 30 && WiFi.status() != WL_CONNECTED; i++) {
        delay(500); Serial.print(".");
    }
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("\nWiFi грешка!");
        while (true) { digitalWrite(PIN_LED_RED, !digitalRead(PIN_LED_RED)); delay(100); }
    }
    Serial.printf("\nIP: %s\n", WiFi.localIP().toString().c_str());

    ledcWriteTone(BUZZER_CHANNEL, 880);
    delay(100);
    ledcWriteTone(BUZZER_CHANNEL, 0);
    Serial.println("Готов.");
}

```

Фиг. 7. setup() функция на ESP32

1.4. Основен цикъл

На фиг. 8 е показаната функцията `loop()`, която се изпълнява циклично. Всяка итерация извиква `inListPassiveTarget()`, която блокира изпълнението докато не засече NFC устройство или изтече вътрешен timeout. При засечен телефон се изпраща SELECT AID APDU чрез `inDataExchange()`. Успешният отговор се парсва и препраща към бекенд сървър, след което се изпраща ACK команда към телефона.

```
void loop() {
    if (!nfc.inListPassiveTarget()) {
        delay(100); // Кратка пауза, за да не претоварваме процесора
        return;
    }
    Serial.println("Засечен е телефон");
    uint8_t response[64];
    uint8_t responseLen = sizeof(response);
    bool success = nfc.inDataExchange(
        (uint8_t*)SELECT_AID_APDU, sizeof(SELECT_AID_APDU),
        response, &responseLen
    );
    if (!success) {
        Serial.println("NFC Транзакцията се провали хардуерно.");
        signalError(); // Директна сигнализация за грешка
        delay(1000); // Пауза, за да може студентът да си дръпне телефона
        return;
    }
    String payload = parsePayload(response, responseLen);

    if (payload.length() > 0) {
        Serial.println("Успешно декодиран пакет: " + payload);
        uint8_t ackCommand[] = {0x00, 0x40, 0x00, 0x00};
        uint8_t ackResponse[16]; // по-голям буфер за всеки случай
        uint8_t ackResponseLen = sizeof(ackResponse);

        bool ackSuccess = nfc.inDataExchange(ackCommand, sizeof(ackCommand), ackResponse, &ackResponseLen);

        if (ackSuccess) {
            Serial.println("Телефонът потвърди получаването на данните (ACK).");
        } else {
            Serial.println("Потвърждението към телефона се провали.");
        }
        sendToServer(payload);
        delay(3000); // Голяма пауза след успех
    } else {
        Serial.println("Грешка при парсване на данните (невалиден формат).");
        signalError();
        delay(1000);
    }
}
```

Фиг. 8. `loop()` функция, чакаща NFC устройство и получаване на `payload` пакет

1.5. Декодиране на получения `payload`

Функцията `parsePayload()` декодира бинарния отговор от Android HCE и реконструира текстовия низ във формат "ФН|АТС|hmachex" за FastAPI. Бинарният

формат е проектиран с length-prefix за факултетния номер, за да се поддържат факултетни номера с различна дължина и водещи нули. Преди обработка се извършват две защитни проверки срещу препълване на буфера. Функцията е на фиг. 9.

```
String parsePayload(uint8_t* response, uint8_t responseLen) {
    if (responseLen < 38) { // Минимален възможен размер (1 + 1 за ФН + 4 + 32)
        Serial.println("Грешка: Пакетът е твърде къс.");
        return "";
    }
    // 1. Четем първия байт, за да разберем дължината на факултетния номер
    uint8_t fnLength = response[0];
    // Защитна проверка за препълване на буфера
    if (1 + fnLength + 4 + 32 > responseLen) {
        Serial.println("Грешка: Несъответствие в дължината на пакета.");
        return "";
    }
    // 2. Извличаме факултетния номер като текст (запазваме водещите нули!)
    String facultyNumber = "";
    for (int i = 0; i < fnLength; i++) {
        facultyNumber += (char)response[1 + i];
    }
    // 3. Намираме къде започва АТС (веднага след ФН)
    int atcOffset = 1 + fnLength;
    uint32_t atc = (response[atcOffset] << 24) |
        (response[atcOffset + 1] << 16) |
        (response[atcOffset + 2] << 8) |
        response[atcOffset + 3];
    // 4. Намираме къде започва HMAC (веднага след АТС)
    int hmacOffset = atcOffset + 4;
    String hmacHex = "";
    for (int i = 0; i < 32; i++) {
        char buf[3];
        sprintf(buf, "%02x", response[hmacOffset + i]);
        hmacHex += buf;
    }
    // 5. Сглобяваме финалния string за FastAPI
    String finalPayload = facultyNumber + "|" + String(atc) + "|" + hmacHex;
    Serial.println("Успешно декодиран динамичен пакет: " + finalPayload);
    return finalPayload;
}
```

Фиг. 9. *parsePayload()* функция за декодиране на бинарния NFC отговор

1.6. Комуникация с бекенда

Функцията `sendToServer()` (на фиг. 10) конструира JSON тяло и изпраща HTTP POST заявка към `/checkin` endpoint-а на FastAPI. Преди заявката се проверява Wi-Fi свързаността, а ако тя се е разпаднала се прави опит за повторно свързване. Времето за

изчакване на отговор от сървъра е 8 секунди, което е достатъчен период за отговор от локален сървър.

Отговорът от сървъра обаче се обработва само до нивото на успешно чекиране или грешка. Тоест, дали проблемът идва от това, че студентът вече се е отчел или сървърът е стигнал до някаква грешка, се отразява само като съобщения в серийния монитор за дебъгване.

```
void sendToServer(const String& nfcPayload) {
    if (WiFi.status() != WL_CONNECTED) {
        WiFi.reconnect(); delay(3000);
        if (WiFi.status() != WL_CONNECTED) { signalError(); return; }
    }

    HTTPClient http;
    http.begin(SERVER_URL);
    http.addHeader("Content-Type", "application/json");
    http.setTimeout(8000);

    JsonDocument doc;
    doc["payload"] = nfcPayload;
    doc["room_number"] = ROOM_NUMBER;
    String body;
    serializeJson(doc, body);

    int code = http.POST(body);
    String resp = http.getString();
    http.end();

    Serial.printf("HTTP %d: %s\n", code, resp.c_str());
    if (code == 200) signalSuccess(); else signalError();
}
```

Фиг. 9. Изпращане на препакетирания payload към сървъра

2. Android мобилно приложение

Мобилното приложение е написано на Kotlin и се състои от четири класа: MainActivity, HomeActivity, NfcHceService и StudentPrefs. За комуникация с бекенда се използва библиотеката Retrofit2, чийто интерфейс е дефиниран в ApiService.kt.

2.1. Съхранение на данните

Класът StudentPrefs отговаря за съхранението на чувствителните данни на студента: JWT токена, HMAC ключа, факултетния номер и АТС брояча. Данните се

записват криптирани на устройството чрез EncryptedSharedPreferences. За управление на ключовете се използва Android Keystore, което означава, че данните не могат да бъдат прочетени дори при root достъп. Разписаният клас е показан на фиг. 10.

```
class StudentPrefs(context: Context) {  
  
    private val masterKey = MasterKey.Builder(context)  
        .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)  
        .build()  
  
    private val prefs = EncryptedSharedPreferences.create(  
        context,  
        "student_secure_prefs",          // Файл с криптирани данни  
        masterKey,  
        EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,  
        EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM  
    )  
  
    var token: String?  
        get() = prefs.getString(KEY_TOKEN, null)  
        set(v) = prefs.edit().putString(KEY_TOKEN, v).apply()  
  
    var hmacKey: String?  
        get() = prefs.getString(KEY_HMAC, null)  
        set(v) = prefs.edit().putString(KEY_HMAC, v).apply()  
  
    var facultyNumber: String?  
        get() = prefs.getString(KEY_FN, null)  
        set(v) = prefs.edit().putString(KEY_FN, v).apply()  
  
    var atcCounter: Long  
        get() = prefs.getLong(KEY_ATC, 0L)  
        set(v) = prefs.edit().putLong(KEY_ATC, v).apply()  
  
    val isProvisioned: Boolean  
        get() = hmacKey != null && facultyNumber != null  
  
    ...  
}
```

Фиг. 10. Клас *StudentPrefs* за криптирано съхранение на данни

Функцията `getAndIncrementAtc()` в класа увеличава АТС брояча с 1 и връща новата стойност. Тя е маркирана с `@Synchronized`, за да се гарантира, че две нишки не могат да я изпълнят едновременно.

АТС броячът се използва основно за защита срещу Replay атаки. Но също така и защитава от студенти, които се опитват да си създадат заявка, с която да отчетат присъствие на занятие без да са присъствали физически. Всъщност, ако някой успее да

открадне данните от заявката при изпращане по NFC, то той ще вземе текущия брояч, но докато успее да създаде нова заявка, и то при евентуална кражба и на НМАС ключа, броячът вероятно вече ще се е увеличил и бекенд сървъра ще откаже присъствието. Ако изпратения АТС брояч е по-малък от този, записан в базата данни на сървъра, то той ще върне грешка. По подобен начин се правят проверки и при разплащане с дебитни карти, примерът за АТС брояч е взет именно от там.

2.2. Комуникация с бекенд сървър

ApiService.kt съдържа дефиницията на всички заявки към сървъра. Използва се библиотеката Retrofit2, при която всяка заявка се описва като функция с анотация. Библиотеката автоматично изпраща заявката и преобразува JSON отговора в Kotlin обект. Дефинирани са две заявки: login за вход в системата и provisionKey за получаване на НМАС ключа от сървъра. Интерфейсът е показан на фиг. 11. Обектът RetrofitClient създава единствена инстанция на клиента, която се използва навсякъде в приложението.

```
data class TokenResponse(  
    val access_token: String,  
    val token_type: String  
)  
data class ProvisionKeyResponse(  
    val hmac_key: String,  
    val faculty_number: String,  
    val atc: Long  
)  
interface ApiService {  
    @FormUrlEncoded  
    @POST("auth/login")  
    suspend fun login(  
        @Field("username") username: String,  
        @Field("password") password: String  
    ): TokenResponse  
  
    @POST("auth/provision-key")  
    suspend fun provisionKey(  
        @Header("Authorization") bearerToken: String  
    ): ProvisionKeyResponse  
}
```

Фиг. 11. Интерфейс ApiService със заявки за логин и provisionKey

Взимането на НМАС ключа се случва само веднъж – при първоначалното логване на студента. Тогава освен ключа се взимат и АТС брояча и факултетния номер от базата данни на сървъра. Ако на мобилно устройство, в което вече има логнат студентски

акаунт, друг се опита да влезе, то при самото влизане с друг акаунт се вземат съответните за студента HMAC ключ, АТС брояч и ФН.

2.3. Клас MainActivity

MainActivity се стартира при отваряне на приложението и играе ролята на логин страница. Вместо обаче приложението да изисква влизане с име и парола всеки път, е създаден механизъм за проверка дали студентът вече се е логвал и има записани данни чрез isProvisioned. Ако да, приложението директно отваря HomeActivity без да показва екрана за вход.

```
lifecycleScope.launch {
    try {
        // Стъпка 1: Логин с вземаме JWT токен
        val tokenResp = RetrofitClient.api.login(username, password)
        prefs.token = tokenResp.access_token

        // Стъпка 2: Даване на HMAC ключ
        val keyResp = RetrofitClient.api.provisionKey("Bearer ${tokenResp.access_token}")

        // Записваме всичко криптирано в Keystore
        prefs.hmacKey = keyResp.hmac_key
        prefs.facultyNumber = keyResp.faculty_number
        // Взимаме АТС-а от сървъра за синхронизация
        // при преинсталация или смяна на телефон
        prefs.atcCounter = keyResp.atc
        getSharedPreferences("nfc_fast_prefs", Context.MODE_PRIVATE)
            .edit()
            .putString("hmac_key", keyResp.hmac_key)
            .putString("faculty_number", keyResp.faculty_number)
            .apply()
        Toast.makeText(this@MainActivity, "Успешен вход!", Toast.LENGTH_SHORT).show()
        goHome()
    } catch (e: HttpException) {
        ...
    }
    ...
}
```

Фиг. 12 Вход и получаване на данни от сървъра

Ако не е намерена информация за логнат потребител, се показва страницата за влизане в профил. На фиг. 12 е показан кода за логването в приложението. След като бъдат въведени данните за вход и се натисне бутона за влизане, се извършват две последователни заявки към сървъра - първо login за получаване на JWT токен, а след това provisionKey за получаване на HMAC ключа, факултетния номер и текущата стойност на

АТС брояча. Заявките се изпълняват асинхронно, за да не блокират интерфейса. При грешка се изписва съобщение според кода на грешката.

2.4. Клас HomeActivity

HomeActivity е главният екран на приложението. Показва факултетния номер на студента и статуса на NFC(показва дали телефонът притежава NFC модул и дали е включен от настройките). На екрана се визуализират GIF анимации спрямо текущото състоянието на NFC модула и стъпката до която е стигнало самото чекиране. Определени са четири състояния, които влизат от едно в друго: NFC изключен, готов за чекиране, активна сесия и резултат от чекирането. Анимациите се зареждат чрез библиотеката Glide.

При натискане на бутона за чекиране се стартира 30-секунден таймер. Полето isCheckinActive в NfcHceService се задава на true, което позволява на услугата да отговаря на NFC четеца (фиг. 13). Ако таймерът изтече преди успешно чекиране, се показва анимацията за неуспех.

```
private fun startCheckinSession() {
    if (nfcAdapter?.isEnabled != true) {
        Toast.makeText(this, "Моля, включете NFC!", Toast.LENGTH_LONG).show()
        return
    }
    NfcHceService.isCheckinActive = true
    btnCheckin.isEnabled = false
    tvCountdown.visibility = View.VISIBLE
    loadGif(R.drawable.nfc_signal)
    countdownTimer?.cancel()
    countdownTimer = object : CountDownTimer(30000, 1000) {
        override fun onTick(millisUntilFinished: Long) {
            tvCountdown.text = "Остават: ${millisUntilFinished / 1000} сек."
            tvNfcStatus.text = "✅ NFC е активен!\nДопете телефона до четеца."
            tvNfcStatus.setTextColor(getColor(android.R.color.holo_green_dark))
        }
        override fun onFinish() {
            // Ако таймерът изтече без успех → показваме грешка
            if (NfcHceService.isCheckinActive) {
                showUnsuccessfulCheckin()
            }
        }
    }.start()
}

private fun showUnsuccessfulCheckin() {
    runOnUiThread {
        loadGif(R.drawable.unsuccessfull_checking)
        tvNfcStatus.text = "❌ Чекирането не бе успешно.\nОпитайте отново."
        tvNfcStatus.setTextColor(getColor(android.R.color.holo_red_dark))

        ivStatusGif.postDelayed({
            resetCheckinUI()
        }, 3000)
    }
}
```

Фиг. 13. Стартиране на сесия за чекиране

Методите `onResume()` и `onPause()` се извикват при показване и излизане (или скриване) на екрана. В `onResume()` се задават два callback-а към `NfcHceService`: единият за успешно чекиране, а другият за грешка. В `onPause()` таймерът се спира, сесията се деактивира и callback функциите се изчистват.

2.5. Клас `NfcHceService`

`NfcHceService` наследява `HostApduService` и реализира виртуалната NFC карта. [10] Това е и може би най-важният и интересен компонент на приложението. Услугата работи при отворено приложение и активиран таймер за чекиране (натиснат е бутоната „Чекирай се“ при което започва да тече 30-секундния диапазон). Три полета в `companion` object управляват комуникацията с `HomeActivity`. Единият от тях е `isCheckinActive`. Той контролира дали услугата отговаря на четеца. При успешно чекиране се извиква `tapCallback`, а `failureCallback` при прекръсната NFC връзка.

Методът `processCommandApdu()` се извиква от Android OS при всяка входяща APDU команда. Първата проверка е за `isCheckinActive` - ако сесията не е активна, се връща `SW_NOT_FOUND` и не изпраща данни. Това е защита против изпращането на информация към чужди четци, които с правилния AID биха успели да извлекат информацията на студента (макар и факултетния номер да е публична информация). Метода е показан на фиг. 14.

```
override fun processCommandApdu(apdu: ByteArray, extras: Bundle?): ByteArray {
    val apduHex = apdu.toHexString()
    if (!isCheckinActive) {
        Log.w(TAG, "Опит за докосване, но режимът е изключен. Игнорираме.")
        return SW_NOT_FOUND
    }
    return when {
        // четеца иска данни
        isOurSelectAid(apdu) -> {
            handleSelectAid()
        }
        // четеца потвърждава (ACK), че е получила данните успешно
        apdu.size >= 2 && apdu[1] == 0x40.toByte() -> {
            // Деактивираме режима и казваме на UI да покаже успех
            isCheckinActive = false
            tapCallback?.invoke()
            SW_OK
        }
        else -> {
            SW_UNKNOWN
        }
    }
}
```

Фиг. 14. Обработка на входящи APDU команди

Методът `isOurSelectAid()` от фиг. 14 проверява дали входящата APDU команда е SELECT AID с точно нашия AID идентификатор. Проверката е байт по байт — първо header-ът (CLA, INS, P1, P2), след това Lc полето и накрая самият AID.

Функцията `handleSelectAid()` (на фиг. 15) чете данните от `StudentPrefs`, изчислява HMAC-SHA256 подписа и конструира бинарния буфер. HMAC се изчислява върху стринга "ФН|АТС" с ключа от `Keystore`.

Буферът съдържа четири части: един байт за дължината на факултетния номер, байтовете на самия номер, четири байта за АТС брояча и 32 байта за HMAC подписа. В края се добавят статус байтовете `SW_OK`.

Използва се `java.nio.ByteBuffer` с `putInt()`, който автоматично записва 4-байтовото цяло число във формат Motorola (байтовете се записват от най-значимия към най-малко значимия). Това е и форматът, който функцията `parsePayload()` в ESP32 очаква.

```
private fun handleSelectAid(): ByteArray {
    val prefs = StudentPrefs(this)
    val facultyNumber = prefs.facultyNumber ?: return SW_NOT_FOUND
    val hmacKeyHex = prefs.hmacKey ?: return SW_NOT_FOUND
    val atc = prefs.getAndIncrementAtc()

    // 1. Факултетен номер в байтове (запазва водещите нули)
    val fnBytes = facultyNumber.toByteArray(Charsets.UTF_8)
    val fnLength = fnBytes.size

    // 2. HMAC-SHA256 върху "ФН|АТС"
    val message = "$facultyNumber|$atc"
    val keyBytes = hmacKeyHex.chunked(2).map { it.toInt(16).toByte() }.toByteArray()
    val mac = Mac.getInstance("HmacSHA256")
    mac.init(SecretKeySpec(keyBytes, "HmacSHA256"))
    val hmacBytes = mac.doFinal(message.toByteArray(Charsets.UTF_8)) // 32 байта

    // 3. Буфер: [1 байт дължина][ФН][4 байта АТС big-endian][32 байта HMAC]
    val buffer = ByteBuffer.allocate(1 + fnLength + 4 + 32)
    buffer.put(fnLength.toByte())
    buffer.put(fnBytes)
    buffer.putInt(atc.toInt())
    buffer.put(hmacBytes)

    return buffer.array() + SW_OK
}
```

Фиг. 15. Създаване на криптографски подписан NFC payload

Методът `onDeactivated()` се извиква от Android при прекъсване на NFC връзката. Ако сесията за чекиране все още е активна при `DEACTIVATION_LINK_LOSS`, това означава, че телефонът се е отдалечил преди получаване на ACK. В този случай се извиква `failureCallback` и `HomeActivity` показва анимация за неуспех.

3. Сървърна част

Сървърната част е монолитна, т.е. отчитането на присъствията и уеб часста се хостват на един сървър. Написана е на езика Python заедно с фреймуърка FastAPI и е организирана в шест основни файла:

- `auth.py` – вход и управление на потребители,
- `attendance.py` – чекиране и управление на присъствия
- `administration.py` – управление на дисциплини, разписания и групи)
- `main.py` – конфигурира приложението и стартира базата данни.
- `models.py` – Разписани са моделите на базата данни
- `shemas.py` – описание на формата на входящите и изходящите данни за всеки endpoint.

3.1. `models.py`

Файлът `models.py` съдържа дефинициите на таблиците в базата данни като Python класове. Всяка таблица наследява класа `Base` от `SQLAlchemy`. При стартиране на приложението функцията `init_models()` проверява дали съществува администраторски акаунт и го създава, ако липсва.

Таблицата `Students` съдържа двете полета, специфични за NFC системата: `hmac_key` за HMAC ключа и `atc_counter` за АТС брояча. АТС брояча е от тип `BigInteger`, за да не се препълни при интензивна употреба. Паролите се съхраняват като `bcrypt` хеш чрез `passlib`

Таблицата `Schedules` съдържа информация за разписанието. Полето `is_biweekly` указва дали занятието се провежда всяка или всяка втора седмица. Полето `subgroup` е незадължително и се използва за подгрупи А и Б при упражнения.

Подробно описание на таблиците се намира в [глава III, точка 5](#).

3.2. `auth.py`

Файлът `auth.py` съдържа логиката за вход, проверка на JWT токени и управление на потребители. JWT токените се издават с 24-часов живот и съдържат потребителското име, ролята и вътрешния идентификатор. Логиката за вход (на фиг. 16) всъщност се активира когато пристигне HTTP POST заявка към адрес `/auth/login`. Префиксът `/auth` се добавя в `main.py` при регистрацията на рутера.


```
@router.post("/login", response_model=Token)
async def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: AsyncSession = Depends(get_async_session),
):
    user = await authenticate_user(db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=401, detail="Грешно потребителско име или парола.")
    token = create_access_token({"sub": user.username, "role": user.role, "user_id": user.id})
    return {"access_token": token, "token_type": "bearer"}
```

Фиг. 16. Endpoint за вход и издаване на JWT токен

Функцията `get_current_user()` проверява JWT токена при всяка заявка. Тя декодира токена, извлича потребителското име и го търси в базата данни. При невалиден или изтекъл токен връща HTTP 401. Функцията е на фиг. 17.

```
async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_async_session),
) -> User:
    exc = HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Невалиден токен.")
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username = payload.get("sub")
        if username is None:
            raise exc
    except JWTError:
        raise exc
    res = await db.execute(select(User).where(User.username == username))
    user = res.scalar_one_or_none()
    if user is None:
        raise exc
    return user
```

Фиг. 17. Проверка на JWT токен при всяка заявка

Endpoint-ът `/auth/provision-key` (на фиг. 18) генерира HMAC ключ за студента при първо извикване. При следващо извикване, например след преинсталация на приложението, връща съществуващия ключ и текущата стойност на АТС брояча от базата данни за синхронизация.

Функциите `require_role()` и `require_any_role()` ограничават достъпа до определени endpoints само за конкретни роли. Администраторът има достъп до всички endpoints независимо от ролята:

```

@router.post("/provision-key", response_model=ProvisionKeyOut)
async def provision_key(
    db: AsyncSession = Depends(get_async_session),
    current_user: User = Depends(get_current_user),
):
    if current_user.role != "student":
        raise HTTPException(status_code=403, detail="Само студенти могат да получат NFC ключ.")

    if not current_user.linked_student_id:
        raise HTTPException(status_code=400, detail="Акаунтът не е свързан с профил на студент.")

    student = await db.get(Student, current_user.linked_student_id)
    if not student:
        raise HTTPException(status_code=404, detail="Студентът не е намерен.")

    if not student.hmac_key:
        # Генерираме 32 криптографски случайни байта (256-bit ключ)
        student.hmac_key = secrets.token_hex(32)
        student.atc_counter = 0
        await db.commit()
        await db.refresh(student)

    return ProvisionKeyOut(
        hmac_key = student.hmac_key,
        faculty_number = student.faculty_number,
        atc = student.atc_counter,
    )

```

Фиг. 18. генериране на HMAC ключ

Освен това във файла са имплементирани и три endpoint-а за регистрация, достъпни само за администратори: /auth/register/student, /auth/register/teacher и /auth/register/admin.

При регистрация на студент се извършват няколко проверки: дали потребителското име вече е заето и дали факултетният номер вече съществува. Ако посочената група не съществува, тя се създава автоматично. Накрая се създават два записа - един в таблица Students и един в таблица Users, свързани чрез linked_student_id

При регистрация на преподавател проверява дали потребителското име е свободно. След това създава два записа: един в таблица Teachers (име, звание, катедра) и един в таблица Users, свързани чрез linked_teacher_id. За разлика от студента, тук не се създава група. Функция за регистрация на преподавател е на фиг. 19, а регистрацията на студент е аналогична.

Регистрацията на администратор е най-простата. Създава само един запис в таблица Users с роля "admin". Няма свързан студент или преподавател. Само

администратор може да регистрира нови потребители, поради това системата няма как да работи в реална учебна среда без администраторски акаунти.

```
@router.post("/register/teacher", dependencies=[Depends(require_role("admin"))])
async def register_teacher(payload: TeacherRegisterIn, db: AsyncSession = Depends(get_async_session)):
    if (await db.execute(select(User).where(User.username == payload.username))).scalar_one_or_none():
        raise HTTPException(status_code=400, detail="Потребителското име вече е заето.")
    try:
        new_teacher = Teacher(name=payload.name, department=payload.department, title=payload.title)
        db.add(new_teacher)
        await db.flush()
        new_user = User(
            username = payload.username,
            password_hash = User.hash_password(payload.password),
            role = "teacher",
            linked_teacher_id = new_teacher.id,
        )
        db.add(new_user)
        await db.commit()
        return {"detail": f"Преподавателят {payload.name} е регистриран успешно!"}
    except Exception as e:
        await db.rollback()
        raise HTTPException(status_code=500, detail=f"Системна грешка: {str(e)}")
```

Фиг. 19. Функция за регистриране на нов преподавател

Има и два endpoint-а за обновяване на профилите при нужда в бъдеще – например смяна на имената или други данни съответно на студента или преподавателя. Това са PATCH заявките /auth/students/{student_id} и /auth/teachers/{teacher_id}

3.3. attendance.py

Файлът attendance.py съдържа логиката за NFC чекиране и управление на присъствията. Основният endpoint е /checkin, който се извиква от ESP32 след прочитане на payload-а от телефона.

Функцията _verify_hmac() изчислява очаквания HMAC подпис и го сравнява с получения. Тя изчислява собствения HMAC подпис като използва същата формула като телефона - взима HMAC ключа на студента от базата данни, съставя низа "ФН|АТС" и го подписва с HMAC-SHA256. Резултатът е 64-символен hex низ.

След това сравнява изчисления подпис с подписа, получен от ESP32. При съвпадение заявката е изпратена от телефон с правилен HMAC ключ. За сравнението се използва hmac.compare_digest() вместо "==", тъй като това е по-сигурният начин за сравняване на такъв тип подписи. Имплементацията на функцията е на фиг. 20.

```
def _verify_hmac(hmac_key_hex: str, faculty_number: str, atc: int, signature: str) -> bool:
    """HMAC-SHA256 верификация – идентична с изчислението в Android приложението."""
    key_bytes = bytes.fromhex(hmac_key_hex)
    message = f"{faculty_number}|{atc}".encode("utf-8")
    expected = hmac_lib.new(key_bytes, message, hashlib.sha256).hexdigest()
    # compare_digest предотвратява timing атаки
    return hmac_lib.compare_digest(expected, signature.lower())
```

Фиг. 20. Верификация на HMAC подписа

Endpoint-ът /checkin изпълнява основната функционалност на бекенда, а именно отбелязването на самото присъствие на студента. Алгоритъмът е направен така, че да отчита когато присъствието е редовно (студентът е отишъл на занятие със своята група) и когато е отработване (студентът е на занятие с друга група, която има същото занятие). Освен това има и логика за отчитане на упражнения, които са през седмица и е вкарано разделение на А и Б подгрупи. В допълнение има проверки дали студентът вече е бил на това упражнение (Например ако миналата седмица е бил на упражнение №3 с А подгрупа, но тази седмица се е объркал и е отишъл на същото упражнение №3 но с Б подгрупа, а е очаквал да отиде на упражнение №4. Знае се, че такива обърквания понякога се случват.)

Алгоритъмът за чекирането е съставен от седем основни стъпки: разделяне на payload-a на трите му части, търсене на студента в базата данни, проверяване на АТС брояча, верифициране на HMAC подписа, търсене на активно занятие в залата(която е подадена като информация от чекиращото устройство), проверяване за дублиране на присъствието и накрая, при условие че всички изисквания са изпълнени, записване на присъствието в базата данни.

На фиг. 21 са показани първите три стъпки от алгоритъма. Следва описание на стъпките подред.

1. Разделяне на payload-a – ESP32 изпраща към сървъра стринг във формат "факултетен_номер|АТС|hmac", например "2301234|42|a3f9bc1d...". Сървърът разделя този низ по символа | и получава три отделни части: факултетен номер, АТС стойност и HMAC подпис. АТС се преобразува от текст към цяло число. Ако стрингът не може да се раздели правилно, сървърът връща грешка HTTP 400.
2. Намиране на студента – Сървърът търси в таблица Students запис с факултетен номер, съвпадащ с получения. Ако не намери студент, връща грешка HTTP

404. Ако студентът е намерен, но няма НМАС ключ в базата данни (никога не се е логвал в приложението), връща грешка HTTP 403.

3. Проверка на АТС брояча – Сървърът сравнява получения АТС с последно записания АТС за този студент в базата данни. Проверката се състои в следните стъпки:

- Полученият АТС трябва да е по-голям от записания в базата данни. Ако е равен или по-малък, означава че някой се опитва да изпрати вече използван payload (което пък означава че се случва replay атака) и сървърът връща грешка HTTP 403.
- Разликата между получения и записания в базата АТС не трябва да надвишава 100 (MAX_ATC_WINDOW). Тази граница съществува защото телефонът увеличава АТС при всяко докосване до четеца, дори ако сървърът не е отговорил успешно. С времето може да се натрупа разлика между АТС на телефона и АТС в базата данни. Тази граница компенсира потенциалните неуспешни опити, при които телефонът е увеличил АТС, но сървърът не е записал присъствие. При разлика по-голяма от 100, се приема, че действието е нерегламентирано и връща грешка HTTP 403.

```
@router.post("/checkin")
async def checkin(payload: CheckinIn, db: AsyncSession = Depends(get_async_session)):
    # 1. Парсиране на payload-a
    try:
        parts = payload.payload.strip().split("|")
        if len(parts) != 3:
            raise ValueError("Очаквани са точно 3 части.")
        faculty_number, atc_str, signature = parts
        atc = int(atc_str)
    except (ValueError, IndexError):
        raise HTTPException(
            status_code=400,
            detail="Невалиден формат на NFC payload. Очакван: 'ФН|АТС|НМАС'."
        )

    # 2. Намиране на студента по факултетен номер
    res = await db.execute(select(Student).where(Student.faculty_number == faculty_number))
    student = res.scalar_one_or_none()
    if not student:
        raise HTTPException(status_code=404, detail="Студентът не е намерен.")

    if not student.hmac_key:
        raise HTTPException(
            status_code=403,
            detail="Няма регистриран NFC ключ. Влезте в приложението за да получите ключ."
        )

    # 3. Replay защита – АТС трябва да е строго по-голям от последния записан
    if atc <= student.atc_counter:
        raise HTTPException(
            status_code=403,
            detail=f"Невалиден АТС ({atc} ≤ {student.atc_counter}). Повторно изпращане?"
        )

    if atc > student.atc_counter + MAX_ATC_WINDOW:
        raise HTTPException(
            status_code=403,
            detail=f"Десинхронизация на АТС! Разликата е твърде голяма ({atc} > {student.atc_counter} + {MAX_ATC_WINDOW})."
```

Фиг. 21 Първите три стъпки от алгоритъма за чекиране

4. Следваща стъпка е верифицирането на HMAC подписа, което се случва чрез извикване на функцията `_verify_hmac()`. Ако проверката не мине успешно, сървърът връща HTTP грешка 403. Кодът за тази стъпка е на фиг. 22.

```
# 4. Верификация на HMAC подписа
if not _verify_hmac(student.hmac_key, faculty_number, atc, signature):
    raise HTTPException(status_code=403, detail="Невалиден криптографски подпис.")
```

Фиг. 22. Проверка на HMAC подписа

5. Алгоритъмът продължава със търсене в графика за занятие, което се провежда в момента в залата на чекиращото устройство, от което е дошла заявката. Тази част от алгоритъма е показана на фиг. 23.

Сървърът търси в таблица Schedules занятие, което отговаря едновременно на няколко условия: залата съвпада с тази, в която е инсталиран четецът, денят от седмицата съвпада с текущия ден, а текущият час е между началния и крайния час на занятието, и текущата дата е в рамките на периода на провеждане на конкретния учебен предмет(между `start_date` и `end_date`).

Може да се намерят няколко занятия, отговарящи на тези условия, затова резултатите се сортират по начална дата възходящо и се обхождат едно по едно.

При двуседмично занятие се изчислява колко седмици са изминали от началната дата до днес. Ако броят на седмиците е нечетен, занятието се пропуска, тъй като не е седмицата на това занятие. (Тук е важно да се оточни, че при създаването на график за занятие, което се провежда през седмица, е важно да се напише начална дата на занятието, тъй като не всички занятия започват от първа учебна седмица на семестъра. Има занятия, които започват от 2-ра или от 3-та учебна седмица)

За всяко намерено занятие сървърът проверява дали групата на студента съвпада с групата на занятието. Ако съвпада, занятието се запазва като основно и търсенето спира. Ако не съвпада, занятието се запазва като гостуващо.

След обхождането на всички намерени занятия се взима решение: ако е намерено основно занятие, то се използва и присъствието ще се запише като "Присъствие". Ако не е намерено основно занятие, но е намерено такова на

друга група, то се използва и присъствието ще се запише като "Отработване".

Ако не е намерено нито едно занятие, сървърът връща грешка HTTP 404.

```
now = datetime.datetime.now()
now_date = now.date()
now_time = now.time()
stmt = (
    select(Schedule)
    .options(joinedload(Schedule.group_course))
    .where(
        and_(
            Schedule.room_number == payload.room_number,
            Schedule.day_of_week == now_date.weekday(),
            Schedule.start_date <= now_date,
            Schedule.end_date >= now_date,
            Schedule.start_time <= now_time,
            Schedule.end_time > now_time,
        )
    )
    .order_by(Schedule.start_date.asc())
)
res_sched = await db.execute(stmt)
schedules = res_sched.scalars().all()
matched_schedule = None
guest_schedule = None
for sched in schedules:
    if sched.is_biweekly:
        weeks_delta = (now_date - sched.start_date).days // 7
        if weeks_delta % 2 != 0:
            continue
    if sched.group_course.group_id == student.group_id:
        matched_schedule = sched
        break
    else:
        guest_schedule = sched
if guest_schedule and not matched_schedule:
    matched_schedule = guest_schedule # Отработване в чужда група
if not matched_schedule:
    raise HTTPException(status_code=404, detail="В момента няма активно занятие в тази зала.")
```

Фиг. 23. Намиране на занятие в залата и

логика за присъствие/отработване

6. Сървърът проверява дали студентът вече се е чекирал за същия предмет и същия тип занятие в рамките на текущия ден. (фиг. 24)

Проверката не е само по schedule_id, а се проверява и предмета и типа занятие.

Ако например един студент се опита да се чекира два пъти за една и съща лекция по Математика, но в различни зали или при различни записи в разписанието, системата пак ще го засече като дублиране.

Ако такъв запис бъде намерен в базата данни, сървърът връща грешка HTTP 403 със съобщение "Вече сте се отчели за този урок днес." и не записва повторно присъствие.

```
# 6. Проверка за дублиране за СЪЩИЯ предмет и тип днес
stmt_dup = (
    select(Attendance)
    .join(Schedule, Attendance.schedule_id == Schedule.id)
    .join(GroupCourse, Schedule.group_course_id == GroupCourse.id)
    .where(
        and_(
            Attendance.student_id == student.student_id,
            GroupCourse.course_id == matched_schedule.group_course.course_id,
            GroupCourse.type == matched_schedule.group_course.type,
            func.date(Attendance.timestamp) == now_date,
        )
    )
)
if (await db.execute(stmt_dup)).scalars().first():
    raise HTTPException(status_code=403, detail="Вече сте се отчели за този урок днес.")
```

Фиг. 24. Проверка за дублиране на присъствието

7. Сървърът определя статуса на присъствието като сравнява групата на студента с групата на намереното занятие. Ако съвпадат, статусът е "Присъствие". Ако не съвпадат, статусът е "Отработване".

След това се създава нов запис в таблица Attendance с факултетния номер на студента, идентификатора на занятието, текущия дата и час, статуса на присъствието, а recorded_by се задава като "Автоматичен", за да се разграничи от ръчно въведените присъствия от преподавател или администратор.

Едновременно с това се обновява АТС брояча на студента в таблица Students със стойността, получена от телефона. АТС се обновява само при успешен запис. Ако по някаква причина записът в базата данни не е успешен, АТС не се обновява.

Накрая се извършва commit към базата на двете промени едновременно. Ако commit-ът се провали, се прави rollback и сървърът връща грешка HTTP 400. ESP32 сигнализира с включване на червен светодиод и звуков сигнал.

При успех сървърът връща HTTP 200 със съобщение "Успешно: Присъствие" или "Успешно: Отработване", което ESP32 получава и съответно сигнализира със светване на зеления светодиод и съответния звуков сигнал. Тази част от алгоритъма е на фиг. 25.


```

# 7. Записване на присъствието
status_text = (
    "Присъствие"
    if student.group_id == matched_schedule.group_course.group_id
    else "Отработване"
)
att = Attendance(
    student_id = student.student_id,
    schedule_id = matched_schedule.id,
    timestamp = now,
    status = status_text,
    recorded_by = "Автоматичен",
)
# Обновяване на АТС брояча (преди да запишем присъствието)
student.atc_counter = atc
db.add(att)
try:
    await db.commit()
except Exception:
    await db.rollback()
    raise HTTPException(status_code=400, detail="Грешка при запис в базата данни.")
return {"detail": f"Успешно: {status_text}", "status": status_text}

```

*Фиг. 25. Записване на присъствието и
обновяване на АТС стойността*

Освен /checkin, файлът attendance.py съдържа още четири endpoint-a.

GET /attendance връща списък с присъствия с търсене и pagination. Ролята на влезлия потребител определя какво вижда - студентите виждат само собствените си записи, преподавателите виждат присъствията на студентите в преподаваните от тях занятия, а администраторите виждат всички присъствия. Резултатът съдържа общия брой записи и текущата страница, което позволява на уеб интерфейса да извежда информацията на отделни страници.

POST /attendance позволява на преподаватели и администратори да добавят присъствие ръчно. Приема студент, занятие, дата и статус. Преди запис се проверява дали преподавателят е свързан с посоченото занятие. Ако не е, получава грешка HTTP 403. Проверява се също дали студентът вече има присъствие за същото занятие на същата дата. За разлика от автоматичното чекиране, тук полето recorded_by се попълва с потребителското име на преподавателя.

PATCH /attendance/{id} променя статуса на съществуващ запис. Преподавателят може да промени само записи, които той самият е създал или са записани автоматично от системата. Не може да се променят записи, въведени от друг преподавател или администратор. Администраторът няма ограничения.

DELETE /attendance/{id} изтрива запис за присъствие. Преподавателят може да изтрие само записи за занятия, които той води, само ако записът е създаден от него или автоматично от системата. Администраторът може да трие всеки запис.

3.4. administration.py

Файлът съдържа endpoint-ите за управление на учебния процес: дисциплини, учебен план, разписание и групи. Всички endpoint-и изискват администраторски права. За всеки обект в системата са реализирани операции за създаване, редакция и в някои случаи изтриване. За пример на фиг. 26 е даден endpoint-а за създаване на дисциплина, като функциите за управление на другите обекти на системата са аналогични и следват същата структура, като се различават по таблицата, към която записват, като проверките са специфични за съответния обект.

```
@router.post("/courses", dependencies=[Depends(require_role("admin"))])
async def create_course(payload: CourseCreate, db: AsyncSession = Depends(get_async_session)):
    # Проверка за дублиране на името
    res_name = await db.execute(select(Course).where(Course.name == payload.name))
    if res_name.scalar_one_or_none():
        raise HTTPException(status_code=400, detail="Дисциплина с това име вече съществува.")

    new_course = Course(name=payload.name)
    db.add(new_course)
    await db.commit()
    return {"detail": f"Дисциплината '{payload.name}' е създадена успешно!"}
```

Фиг. 26. Endpoint за създаване на дисциплина

3.5. main.py

Това е файлът, от който се стартира сървърът. Конфигурира CORS за достъп от браузъра, регистрира трите рутера с техните URL префикси и маунтва статичните файлове на уеб интерфейса. При стартиране автоматично създава таблиците в базата данни. Част от файла е изложена на фиг. 27.

```

app = FastAPI(title="Attendance API")

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# API Routers (задължително ПРЕДИ StaticFiles)
app.include_router(auth_router, prefix="/auth", tags=["auth"])
app.include_router(attendance.router, prefix="", tags=["attendance"])
app.include_router(administration.router, prefix="/admin", tags=["admin"])

# — Статични файлове: CSS, JS, лого —
# Монтира се на /static, но файловете са в текущата директория.
# html=True позволява директен достъп по filename.
app.mount("/static", StaticFiles(directory=".", html=False), name="static")

```

Фиг. 27. Конфигурация на FastAPI в main.py

4. Уеб интерфейс

Уеб интерфейсът е реализиран с Vanilla JavaScript и се състои от три файла - index.html (структура на страницата), styles.css (визия) и app.js (логика). FastAPI доставя тези файлове директно (статично зареждане), без нужда от отделен уеб сървър. Целият код на интерфейса се съдържа в app.js и е разделен на няколко логически части.

4.1. Глобално състояние

Всички данни, заредени от сървъра, се пазят в един глобален обект S (фиг. 28). Той съдържа JWT токена, ролята и потребителското име на влезналия потребител, списъците със студенти, преподаватели, групи, дисциплини, учебен план и разписание, текущата страница за всеки списък и параметрите за търсене в регистъра на присъствията.

```

const S = {
  token: null, role: null, userId: null, username: null,
  groups: [], students: [], teachers: [], courses: [], groupCourses: [], schedules: [],
  att: { skip: 0, limit: 50, total: 0, search: '' },
  cpag: { students:1, teachers:1, groups:1, courses:1, gc:1, schedules:1 },
  filtered: { students:[], teachers:[], groups:[], courses:[], gc:[], schedules:[] },
  PER_PAGE: 40,
  currentSection: 'dashboard',
  modalCallback: null
};

```

Фиг. 28. Глобален обект за съхранение на данните

4.2. Функция api()

Всички заявки към сървъра минават през функцията `api()`, показана на фиг. 29. Тя приема HTTP метод, път, тяло на заявката и незадължителни параметри за търсене. Автоматично добавя JWT токена в заглавието на заявката и при грешка извлича съобщението от отговора на сървъра. Използват се относителни URL адреси, което означава, че интерфейсът работи на всеки адрес без промяна в кода.

```
async function api(method, path, body, queryParams) {
  let url = path;
  if (queryParams) {
    const p = new URLSearchParams();
    for (const [k, v] of Object.entries(queryParams))
      if (v !== null && v !== undefined && v !== '') p.append(k, v);
    const qs = p.toString();
    if (qs) url += '?' + qs;
  }
  const opts = { method, headers: { 'Content-Type': 'application/json' } };
  if (S.token) opts.headers['Authorization'] = 'Bearer ' + S.token;
  if (body !== undefined && body !== null) opts.body = JSON.stringify(body);
  const resp = await fetch(url, opts);
  if (resp.status === 204) return {};
  const data = await resp.json().catch(() => ({}));
  if (!resp.ok) {
    const msg = data.detail || `HTTP ${resp.status}`;
    throw new Error(typeof msg === 'string' ? msg : JSON.stringify(msg));
  }
  return data;
}
```

Фиг. 29. Функция `api()` за комуникация със сървъра

4.3. Вход в системата

При зареждане на страницата се проверява дали в `localStorage` има запазен JWT токен от предишна сесия (фиг. 30). Ако токенът е валиден и не е изтекъл, потребителят директно влиза в системата без да въвежда отново потребителско име и парола.

```
window.addEventListener('load', async () => {
  const saved = localStorage.getItem('tu_token');
  if (saved) {
    const payload = parseJWT(saved);
    if (payload && payload.exp * 1000 > Date.now()) {
      S.token = saved; S.role = payload.role; S.userId = payload.user_id; S.username = payload.sub;
      await initApp(); return;
    }
    localStorage.removeItem('tu_token');
  }
  document.getElementById('login-screen').style.display = 'flex';
});
```

Фиг. 30. Автоматичен вход при зареждане на страницата

Ако токeнът e изтекъл или липсва, се показва лoгин екранът. Функцията doLogin() изпраща потребителското име и паролата към /auth/login. При успех декодира JWT токена, за да извлече ролята и потребителското id, записва токена в localStorage и зарежда главния екран. Функцията parseJWT() декодира средната част на токена, която e Base64 кодирана и съдържа данните на потребителя. Тялото на функцията e показано на фиг. 31.

```
async function doLogin() {
  const user = $v('l-user'), pass = document.getElementById('l-pass').value;
  const errEl = document.getElementById('login-error');
  errEl.style.display = 'none';
  if (!user || !pass) { showLoginErr('Попълнете всички полета.');
```

Фиг. 31. Вход с декодиране на JWT токен

4.4. Навигация и страничен панел

След успешен вход функцията initApp() скрива лoгин екрана, показва основния интерфейс и изгражда страничния панел според ролята на потребителя. Конфигурацията на панела e дефинирана в обекта SIDEBAR_CFG (фиг. 32), който съдържа различни елементи за всяка роля. Администраторът вижда всички секции, преподавателят вижда само разписанието и присъствията в неговите занятия, а студентът вижда само собствените си присъствия.

Функцията nav() на фиг. 33 се извиква при избор на елемент от панела. Тя скрива всички секции, показва избраната и извиква съответната функция за зареждане на данните. Всяка секция има своя render функция, която се търси в обект по id на секцията.

```
const SIDEBAR_CFG = {
  admin: [
    { lbl: 'Общ преглед', items: [{ id: 'dashboard', icon: '📊', label: 'Табло' }] },
    { lbl: 'Потребители', items: [
      { id: 'students', icon: '👤', label: 'Студенти' },
      { id: 'teachers', icon: '👨‍🏫', label: 'Преподаватели' },
      { id: 'admins', icon: '🔑', label: 'Администратори' }
    ] },
    { lbl: 'Учебен процес', items: [
      { id: 'groups', icon: '👥', label: 'Групи' },
      { id: 'courses', icon: '📖', label: 'Дисциплини' },
      { id: 'gc', icon: '📅', label: 'Учебен план' },
      { id: 'schedules', icon: '🕒', label: 'Разписание' }
    ] },
    { lbl: 'Присъствия', items: [{ id: 'attendance', icon: '✅', label: 'Регистър' }] }
  ],
  teacher: [
    { lbl: 'Моите часове', items: [{ id: 'schedules', icon: '🕒', label: 'Разписание' }] },
    { lbl: 'Присъствия', items: [
      { id: 'attendance', icon: '✅', label: 'Регистър' },
      { id: 'add-att', icon: '➕', label: 'Добави ръчно' }
    ] }
  ],
  student: [
    { lbl: 'Моите данни', items: [{ id: 'attendance', icon: '✅', label: 'Моите присъствия' }] }
  ]
};
```

Фиг. 32. Конфигурация на страничния панел по потребителска роля

```
function nav(id) {
  document.querySelectorAll('.section').forEach(s => s.classList.remove('active'));
  document.querySelectorAll('.sb-item').forEach(s => s.classList.remove('active'));
  const sec = document.getElementById('section-' + id);
  if (sec) sec.classList.add('active');
  const ni = document.getElementById('nav-' + id);
  if (ni) ni.classList.add('active');
  S.currentSection = id;
  document.getElementById('content-area').scrollTop = 0;
  renderSection(id);
}

function renderSection(id) {
  ({ dashboard: renderDashboard, students: renderStudents, teachers: renderTeachers,
    groups: renderGroups, courses: renderCourses, gc: renderGC,
    schedules: renderSchedules, attendance: renderAttendance, 'add-att': renderAddAtt
  }[id] || (() => {}))();
}
```

Фиг. 33. Навигация между секциите

4.5. Зареждане на данни

При вход функцията loadRefData() зарежда едновременно всички списъци от сървъра - студенти, преподаватели, групи, дисциплини, учебен план и разписание. Заявките се изпращат едновременно чрез Promise.all(), което намалява времето за зареждане. Студентите не зареждат тези данни, тъй като нямат достъп до тях

4.6. Разделяне на страници

Уеб интерфейсът използва два вида разделяне на страници в зависимост от секцията. За студенти, преподаватели, групи, дисциплини, учебен план и разписание се използва клиентско разделяне. Всички данни се зареждат веднъж при вход и се разделят на страници локално. За регистъра на присъствията се използва сървърно разделяне, тъй като записите могат да бъдат много на брой.

При клиентското разделяне функцията `slicePage()` връща само записите за текущата страница, а `buildPag()` изгражда бутоните за навигация. При търсене страницата се нулира до 1.

За регистъра на присъствията параметрите `skip` и `limit` се изпращат към сървъра при всяка промяна на страницата. Търсенето използва кратко закъснение преди изпращане на заявката, за да не се претоварва сървърът при всяко натиснато копче. Тази част от кода може да бъде видяна на фиг. 34.

```
function debouncedAttSearch() {
  clearTimeout(_attDeb);
  _attDeb = setTimeout(() => { S.att.search = $v('att-search'); S.att.skip = 0; renderAttendance(); }, 350);
}
async function renderAttendance() {
  const canEdit = S.role === 'admin' || S.role === 'teacher';
  document.getElementById('att-add-btn').style.display = canEdit ? '' : 'none';
  document.getElementById('att-actions-th').textContent = canEdit ? 'Дейности' : '';
  const tbody = document.getElementById('att-tbody');
  tbody.innerHTML = '<tr class="empty-row"><td colspan="7">Зареждане...</td></tr>';
  try {
    const data = await api('GET', '/attendance', null, { skip:S.att.skip, limit:S.att.limit, search:S.att.search||undefined });
    S.att.total = data.total;
    tbody.innerHTML = data.items.length
      ? data.items.map(a => `<Код за рендериране>`).join('') //Съкратен вариант за документацията
      : '<tr class="empty-row"><td colspan="7">Няма намерени записи.</td></tr>';
    buildAttPag();
  } catch(e) {
    tbody.innerHTML = `<tr class="empty-row"><td colspan="7" style="color:var(--red)">Грешка: ${e.message}</td></tr>`;
  }
}
```

Фиг. 34. Разделяне на страници на регистъра на присъствия

4.7. Модален прозорец

За операции като добавяне и редакция на записи се използва един общ модален прозорец. Съдържанието му се попълва динамично при отваряне според операцията. Функцията `modalSubmit()` от фиг. 35 записва в `S.modalCallback` и се извиква при натискане на бутона „Запази“. При грешка се показва toast съобщение, а модалният прозорец остава отворен.

```
async function modalSubmit() {  
  if (!S.modalCallback) return;  
  const btn = document.getElementById('modal-submit');  
  btn.disabled = true; btn.textContent = 'Запазване...';  
  try { await S.modalCallback(); closeModal(); }  
  catch(e) { toast(e.message, 'error'); }  
  finally { btn.disabled = false; btn.textContent = 'Запази'; }  
}
```

Фиг. 35. Изпращане на данни от модалния прозорец

4.8. Toast известия

За кратките известия към потребителя се използват toast съобщения, които се появяват в долния десен ъгъл на екрана и изчезват след 5 секунди. Поддържат се три вида - success (зелено), error (червено) и info (синьо).

VI. Ръководство на потребителя

Настоящата глава описва инсталирането и конфигурирането на системата, както и начина на работа с всеки от нейните компоненти. Ръководството е насочено към трите групи потребители, а именно администратори, преподаватели и студенти.

1. Системни изисквания

За инсталиране и работа на системата са необходими следните компоненти:

1. Компютър или сървър с инсталиран Python 3.11 или по-нов за сървърната част
2. Локална Wi-Fi мрежа, към която са свързани сървърът и ESP32 четецът.
3. Android смартфон с NFC поддръжка и Android 5.0 или по-нов за мобилното приложение
4. ESP32-WROOM32 микроконтролер със свързан PN532 NFC четец и свързани към него светодиоди и зумер, както е показано на електрическата схема в [ГЛ. III, т. 2.1](#)
5. Браузър за достъп до уеб интерфейса. Изпробвано и работещо с Google Chrome

2. Инсталиране и конфигуриране

2.1. Инсталиране на сървърната част

Сървърната част се инсталира на компютъра, който ще обслужва системата. Стъпките за инсталация са следните:

1. Изтеглете файловете на проекта (от преносимото устройство, приложено към документацията на дипломната работа) и отворете папката в терминал.
2. Инсталирайте необходимите Python библиотеки с командата:

```
pip install fastapi uvicorn sqlalchemy aiosqlite python-jose passlib[bcrypt] python-multipart
```
3. Стартирайте сървъра с командата:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```
4. При първо стартиране базата данни се създава автоматично и се генерира администраторски акаунт с потребителско име admin и парола admin123. Препоръчително е паролата да се смени при първо влизане (Засега възможно единствено чрез директна промяна през базите данни.

5. Препоръчително е да се зададе статичен IP адрес на компютъра в локалната мрежа (през настройките на маршрутизатора). Той ще се използва за конфигурация на ESP32 и мобилното приложение.

2.2. Конфигурация и флашване на ESP32

Преди качване на фърмуера е необходимо да се зададат мрежовите параметри. Отворете файла config.h и попълнете следните стойности:

```
#define WIFI_SSID "Името на вашата Wi-Fi мрежа"

#define WIFI_PASS "Паролата на вашата Wi-Fi мрежа"

#define SERVER_URL "http://192.168.1.X:8000/checkin"

#define ROOM_NUMBER "101"
```

192.168.1.X се заменя с действителния IP адрес на сървъра, а 101 - с номера на залата, в която е монтиран четецът. След това кодът се флашва на ESP32 чрез PlatformIO с командата Upload. При успешна инициализация четецът издава кратък звуков сигнал.

2.3. Инсталиране на мобилно приложение

Мобилното приложение се инсталира на Android смартфона на студента. Преди инсталиране трябва да се провери дали IP адресът на сървъра в ApiService.kt съответства на действителния адрес. Приложението се компилира и инсталира чрез Android Studio или се разпространява като APK файл.

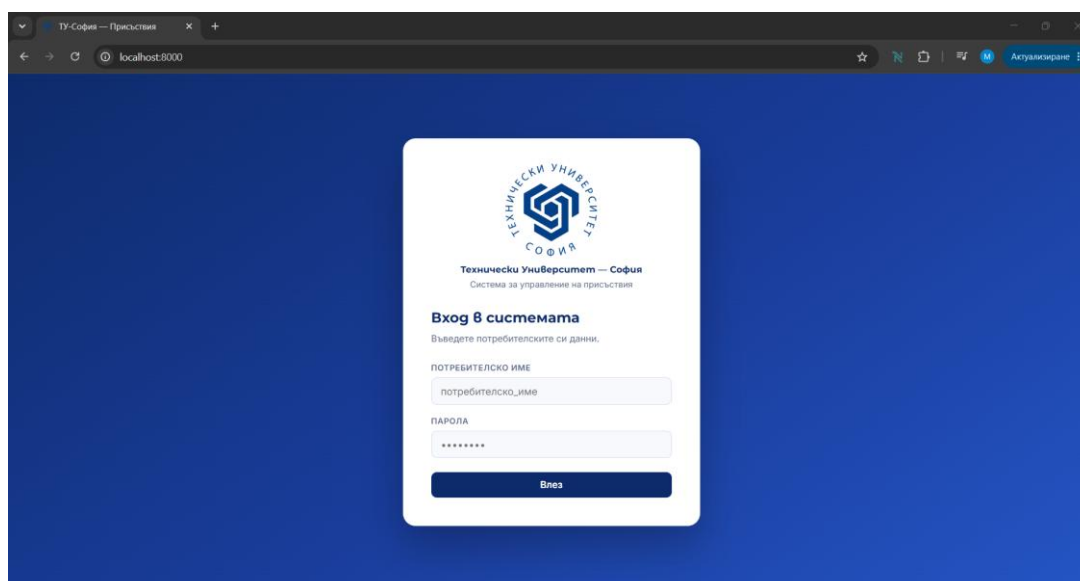
На смартфона трябва да е включен NFC. Ако NFC е изключен, приложението показва предупреждение и предлага да се отиде в настройките за включване.

3. Работа с уеб интерфейса

3.1. Вход в системата

Уеб интерфейсът се достъпва от браузър на адрес http://IP_адрес:8000. При отваряне се показва логин екран с полета за потребителско име и парола, показани на фиг. 36. Използвани са цветовете и логото на ТУ-София.

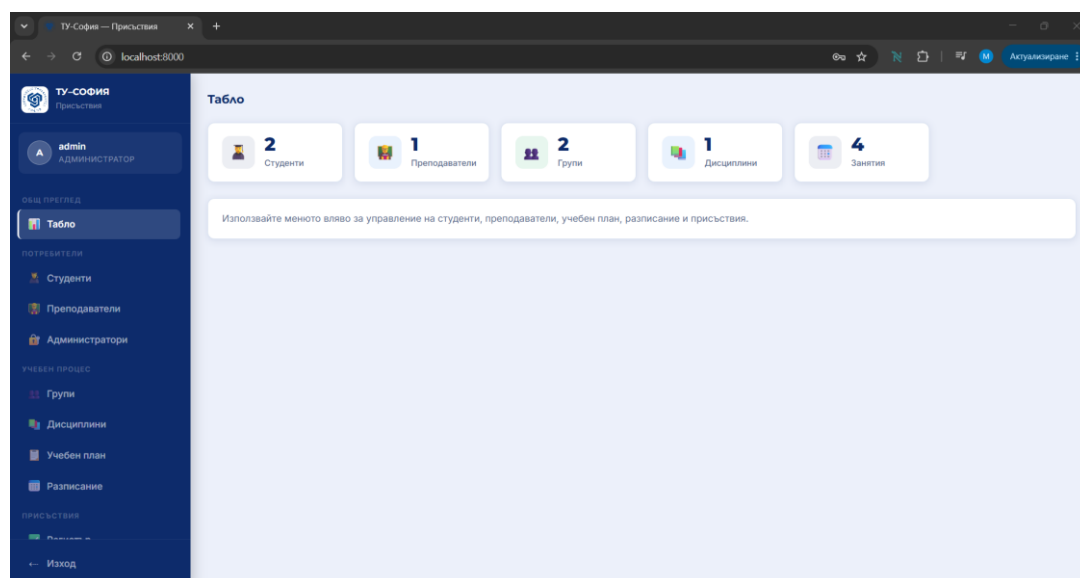
След успешен вход потребителят вижда главния интерфейс с навигационен панел вляво и съдържание вдясно. Навигационният панел се различава според ролята на влезлия потребител.



Фиг. 36. Логин екран в уеб интерфейса

3.2. Работа с администраторски профил

Администраторът има достъп до всички секции на системата. При вход като администратор се показва табло с обобщена статистика (фиг. 37), която включва брой студенти, преподаватели, групи, дисциплини и занятия.



Фиг. 37. Административно табло

Регистрирането на нов студент се извършва от секция Студенти. При натискане на бутона „Нов студент“ се отваря прозорец (фиг. 38) с полета за потребителско име, парола, три имена, факултетен номер и група. Ако посочената група не съществува, тя се създава автоматично.

Регистрация на нов студент

ПОТР. ИМ. ПАРОЛА

ПЪЛНО ИМ. ФАКУЛТ. №

RFID UID ГРУПА

КУРС (1-5) СПЕЦИАЛНОСТ

Ако групата не съществува, ще бъде създадена автоматично.

Фиг. 38. Регистрация на нов студент

За да се създаде разписание (фиг. 39), трябва да се следва тази последователност:

1. Създаване на групи от секция „Групи“ (или автоматично при процеса по добавяне на нови студенти).
2. Създаване на дисциплини от секция „Дисциплини“.
3. Регистриране на преподаватели от секция „Преподаватели“.
4. Задаване на дисциплини към групи от секция „Учебен план“, като се посочва преподавателят и типа на занятието.
5. Добавяне на конкретни часове от секция „Разписание“, като се посочват залата, денят, часът и периодът на провеждане.

Ново занятие

ЗАНЯТИЕ (УЧЕБЕН ПЛАН)

ЗАЛА ДЕН

НАЧ. ЧАС КР. ЧАС

ПОДГРУПА ПОВТОРЯЕМОСТ

НАЧАЛНА ДАТА КРАЙНА ДАТА

Фиг. 39. Добавяне на разписание

3.3. Работа с преподавателски профил

Преподавателят вижда само занятията, за които е отговорен. От секция Разписание може да прегледа своите часове. От секция „Регистър“ може да прегледа присъствията за своите занятия, да промени статуса на запис или да го изтрие.

За ръчно добавяне на присъствие преподавателят избира секция „Добави ръчно“, избира занятие от своето разписание, избира студент от групата на занятието, задава дата и статус и натиска Запиши. Бутоните са отбелязани в червено на фиг. 40.

ТУ-СОФИЯ
Присъствия

ggeorgiev
ПРЕПОДАВАТЕЛ

МОИТЕ ЧАСОВЕ

Разписание

ПРИСЪСТВИЯ

Регистър

+ Добави ръчно

Добави Присъствие Ръчно

Нов запис за присъствие

ЗАНЯТИЕ (ОТ РАЗПИСАНИЕТО)

Kompiutarni mreji [Упражнение] – 3 – Четвъртък 08:00:00–23:30:00 (101)

СТУДЕНТ

Ivan Petrov (ФН: 222222222)

ДАТА

15.06.2026 г.

СТАТУС

Присъствие

Запиши Изчисти

Фиг. 40. Ръчно добавяне на присъствие

3.4. Работа със студентски профил

Студентът вижда само секция Моите присъствия, където са изброени всички негови записи за присъствие с дата, час, дисциплина и статус. Студентът не може да редактира или изтрива записи. Табло на студентски профил е демонстрирано на фиг. 41.

ТУ-СОФИЯ
Присъствия

Ivan
СТУДЕНТ

МОИТЕ ДАННИ

Моите присъствия

Присъствия

Регистър на присъствия

Търси по студент или ФН... 50 / стр.

#	ДАТА И ЧАС	СТУДЕНТ	ДИСЦИПЛИНА	СТАТУС	ВЪВЕДЕН ОТ
2	01.06.2026 г. 21:54	Ivan Petrov (ФН: 222222222)	Kompiutarni mreji (Упражнение)	Отработано	Автоматичен

1-1 от 1 записа 1

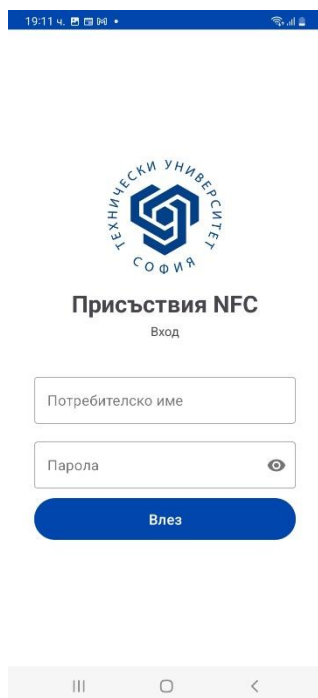
← Изход

Фиг. 41. Присъствия на студент

4. Работа с мобилно приложение

4.1. Първоначално стартиране и вход в приложението

При първо стартиране на приложението се показва логин екран. Студентът въвежда потребителско име и парола, зададени от администратора, и натиска бутона Влез. Приложението се свързва със сървъра, получава НМАС ключа и АТС брояча и ги записва криптирано на устройството. При следващи стартирания логин екранът се прескача автоматично.



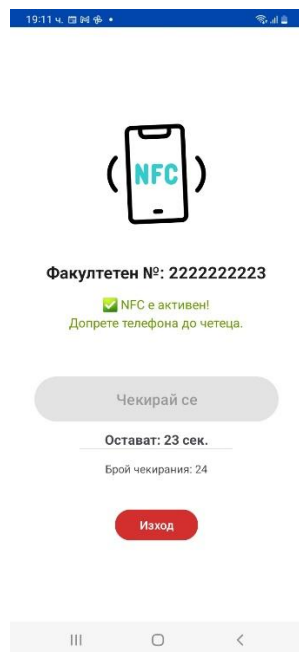
Фиг. 42. Логин екран на мобилното приложение

4.2. Чекиране с NFC

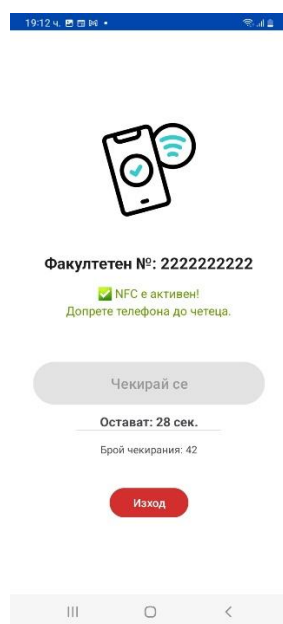
След вход се показва главният екран с факултетния номер на студента и статуса на NFC. За да се чекира студентът, трябва да натисне бутона „Чекирай се“. Приложението активира 30-секунден времеви прозорец и показва анимация, указваща че приложението е готово за четене. Статусът на активирано чекиране и чакане за чекиращо устройство е показан на фиг. 43.

Студентът доближава телефона до NFC четеца в залата. Четецът чете данните, изпраща ги към сървъра и изпраща потвърждение обратно към телефона. При успешно чекиране приложението показва анимация и съобщение за успех. Едновременно с това четецът в залата издава съответния звуков сигнал и включва зеления светодиод за

определен период от време. Екранът на приложението в този етап на чекирането е на фиг. 44.

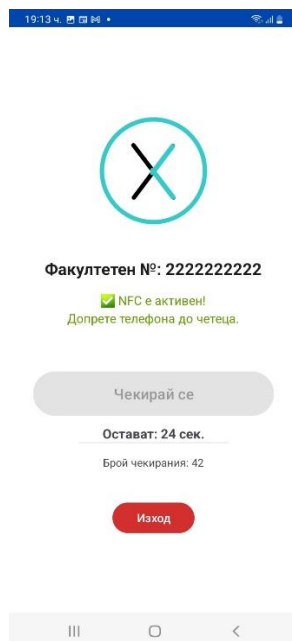


Фиг. 43. Активирана NFC сесия



Фиг. 44. Успешно чекиране

Ако чекирането не е успешно, например поради изтекъл таймер, грешен НМАС подпис или вече записано присъствие за деня, приложението показва червена анимация и съобщение за грешка. Четецът в залата издава нисък звуков сигнал и светва за определен период червеният светодиод. (фиг. 45)



Фиг. 45. Неуспешно чекиране

5. Често срещани проблеми

В *табл. 1* са описани най-честите проблеми при работа с системата и начините за отстраняването им:

Проблем	Възможна причина	Решение
Четецът не реагира на телефона	NFC е изключен на телефона	Включете NFC от Настройките на телефона. Ако вече е включен, доближете телефона на разстояние под 4 см до четеца.
Червен светодиод след допиране	Сървърът е върнал грешка	Проверете дали има активно занятие в залата и дали вече не сте чекирани
Приложението не се свързва при вход	Сървърът не е стартиран или IP адресът е грешен	Проверете дали сървърът работи и дали IP адресът в ApiService.kt е правилен
Червеният светодиод на ESP32 мига при стартиране	PN532 не е намерен или Wi-Fi не е достъпен	Проверете схемата на свързване на PN532 и мрежовите параметри в config.h
АТС грешка при чекиране	Голямо разминаване между АТС на телефона и сървъра	Излезте от приложението и влезте отново за синхронизиране на АТС

Таблица 1 – често срещани проблеми

VII. Заключение

Настоящият дипломен проект разработи и реализира прототип на система за автоматизирано отчитане на студентско присъствие в Технически университет София, базирана на NFC технология и Host Card Emulation. Системата замества ръчния процес на отчитане с решение, при което студентът доближава своя смартфон до четец в залата и присъствието се записва автоматизирано. Тя покрива пълния функционален цикъл - регистрация на студент и получаване на НМАС ключ, автоматично NFC чекиране чрез мобилното приложение, както и уеб интерфейс за администриране. Реализирана е криптографска защита на три нива: верификация на самоличността чрез НМАС подпис, защита срещу повторно изпращане чрез АТС брояч и защита срещу дублирано чекиране.

Разработеният прототип има следните ограничения, които не са обхванати от настоящия проект:

- Системата е тествана в локална мрежова среда. Вкарването на системата в реална университетска среда би изисквало HTTPS връзка, по-надеждна мрежова инфраструктура и интеграция с официалната информационна система на ТУ-София.
- Мобилното приложение поддържа само Android устройства. iOS не дава достъп до Host Card Emulation без нужните сертификати и единствено след одобрение от страна на Apple. За одобрение и разработка на такъв тип приложение е необходим и Apple Developer акаунт. За съжаление сегашната разработка изключва значителна част от потребителите, които използват продукти на Apple.
- При едновременна работа на много четци в различни зали производителността на SQLite може да се окаже ограничение. При по-голям мащаб би се наложила миграция към PostgreSQL.
- Системата не поддържа автоматично обновяване на информацията за студентите (Например обновяване на курс или група през следващия семестър или година). За целта е нужна връзка с официалната информационна система на ТУ-София.

На база на реализирания прототип могат да се развият няколко допълнителни функционалности.

Първата от тях е добавяне на HTTPS с цел по-голяма сигурност при предаването на пакети между устройствата.

Тъй като мобилното приложение поддържа само Android устройства, може да се разработи алтернативен механизъм за iOS, но само след изпълняване на сложните процедури по сертифициране за използване на услугата Wallet от страна на Apple.

Друга възможна посока е добавяне на автоматични известия към студенти и преподаватели при достигане на определен праг на отсъствия. Може да бъде добавено и известяване за студентите през мобилното им приложение, което да напомня за започването на дадено занятие от разписанието им.

Системата може да се интегрира и с официалната информационна система на ТУ-София за автоматично синхронизиране на студенти и разписания. Накрая може да се разработи статистически модул с графики и справки за анализ на присъствията по дисциплина, група и период. Чрез свързването със системата може да се направи автоматично обновяване на информацията за студентите семестър за семестър.

Реализираният прототип демонстрира, че NFC технологията и Host Card Emulation предоставят надеждна и сигурна основа за автоматизирано отчитане на присъствие без нужда от специализиран хардуер от страна на студента. Смартфонът, който студентът вече притежава, е достатъчен за пълноценна работа на системата.

Речник на използвани съкращения и термини

NFC - Near Field Communication (комуникация на близко разстояние)

HCE - Host Card Emulation (емулация на карта от хост)

HMAC - Hash-based Message Authentication Code (код за удостоверяване на съобщение базиран на хеш)

ATC - Application Transaction Counter (брояч на транзакции на приложението)

APDU - Application Protocol Data Unit (единица данни на приложния протокол)

SPI - Serial Peripheral Interface (сериен периферен интерфейс)

JWT - JSON Web Token

AES - Advanced Encryption Standard (усъвършенстван стандарт за криптиране)

API - Application Programming Interface (интерфейс за програмиране на приложения)

REST - Representational State Transfer

ORM - Object-Relational Mapper

SHA - Secure Hash Algorithm (защитен хеш алгоритъм)

GPIO - General Purpose Input/Output (универсален вход/изход)

PWM - Pulse Width Modulation (широчинно-импулсна модулация)

AID - Application Identifier (идентификатор на приложение)

RFID - Radio Frequency Identification (радиочестотна идентификация)

JSON - JavaScript Object Notation

URL - Uniform Resource Locator

HTTP/HTTPS - Hypertext Transfer Protocol (Secure)

Replay атака - атака при която прихванато съобщение се изпраща повторно

Payload - полезното съдържание на предадените данни

Endpoint - адрес на API, към който се изпращат заявки

Фърмуер - програмен код, записан директно в хардуера на микроконтролера

Източници

- [1] T. Arulogun et al., „RFID-Based Students Attendance Management System," International Journal of Engineering and Scientific Research, vol. 4, 2013. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/235598499>
- [2] C. B. Chew et al., „Sensors-enabled Smart Attendance Systems Using NFC and RFID Technologies," ResearchGate, 2015. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/280027652>
- [3] Google, „Google Classroom," Google for Education, 2024. [Онлайн]. Достъпно на: <https://edu.google.com/workspace-for-education/products/classroom/>
- [4] Microsoft, „Microsoft Teams for Education," Microsoft, 2024. [Онлайн]. Достъпно на: <https://www.microsoft.com/en-us/education/products/teams>
- [5] B. Ahmad, M. A. Ayu, „TouchIn: An NFC Supported Attendance System in a University Environment," International Journal of Information and Education Technology, vol. 4, 2014. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/300791064>
- [6] G. P. Mithun et al., „An NFC Based Student Attendance Tracking/Monitoring System Using an IoT Approach," 2020 International Conference on Computational Science and Computational Intelligence (CSCI), 2020. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/352698803>
- [7] M. S. Mane et al., „NFC-based Mobile Application for Student Attendance in Institution of Higher Learning," ResearchGate, 2022. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/363827165>
- [8] N. Hamzah et al., „A Review of Student Attendance System Using Near-Field Communication (NFC) Technology," ResearchGate, 2015. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/300791064>
- [9] O. Olatunji et al., „Development of a smart attendance system using near field communication (SMAT-NFC)," ResearchGate, 2022. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/363195201>
- [10] B. Ozdenizci et al., „A Tokenization-Based Communication Architecture for HCE-Enabled NFC Services," Mobile Information Systems, Hindawi, 2016. [Онлайн]. Достъпно на: <https://hindawi.com/journals/misy/2016/5046284>

[11] Y. Feng et al., „A Replay-Attack Resistant Authentication Scheme for the Internet of Things," ResearchGate, 2017. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/319053602>

[12] A. Maier et al., „Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for the Internet of Things," ResearchGate, 2017. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/320273388>

[13] Adafruit Industries, „Adafruit PN532 NFC/RFID Controller Shield for Arduino + Extras," GitHub, 2024. [Онлайн]. Достъпно на: <https://github.com/adafruit/Adafruit-PN532>

[14] R. Putri et al., „Authenticate and Verification Source Files using SHA256 and HMAC Algorithms," ResearchGate, 2024. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/381942463>

[15] A. Hamid et al., „Implementation of HMAC-SHA256 algorithm for hybrid routing protocols in MANETs," ResearchGate, 2021. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/282265667>

[16] M. Bellare, „New proofs for NMAC and HMAC: Security without collision-resistance," ResearchGate. [Онлайн]. Достъпно на: <https://www.researchgate.net/publication/220334217>

[17] Android Developers, „Android Keystore system," Google, 2024. [Онлайн]. Достъпно на: <https://developer.android.com/privacy-and-security/keystore>

[18] M. Lathkar, „High-Performance Web Apps with FastAPI: The Asynchronous Web Framework Based on Modern Python," Apress/Springer, 2023. [Онлайн]. Достъпно на: <https://link.springer.com/book/10.1007/978-1-4842-9178-8>

[19] Atmel Corporation, „Understanding the Requirements of ISO/IEC 14443 for Type B Proximity Contactless Identification Cards," Microchip, 2005. [Онлайн]. Достъпно на: <https://ww1.microchip.com/downloads/en/devicedoc/doc2056.pdf>

[20] Android Developers, „Host-based Card Emulation overview," Google, 2024. [Онлайн]. Достъпно на: <https://developer.android.com/develop/connectivity/nfc/hce>